DATA & AI PORTFOLIO · PROJECT NOTEBOOK 15

Aegis Health Plan Intelligence Platform

Governed metrics, survival analysis, and a detector that an adversary switched off

Author Nikhil Sinha

Repository github.com/Nikhil201716/15-Aegis-Health-Plan-Platform

Stack Python · DuckDB · scikit-learn · statsmodels · FastAPI · Ollama

Edition 1.0

How to read this book

This book is written to be read from the beginning by someone who has not seen the project and does not yet know the techniques it uses. It is not API documentation - it is closer to a course text built around one real working system.

Two conventions run throughout. Callouts marked **WHY** explain a design decision and name the alternative that was rejected; callouts marked **WARNING** mark a trap that is easy to fall into and hard to notice afterwards. There is a chapter of pure terminal commands so the project can be reproduced without reading anything else, and a chapter of exercises with worked solutions.

Every code listing is extracted from the repository at the moment this book is built, so nothing here can drift out of step with the code it describes. If a listing looks wrong, the code is wrong.

All data in this project is synthetic. No real customer, transaction, or company data appears anywhere.

Contents

CHAPTER 1

The Business Problem

> *Why a health plan needs all five disciplines at once, and what makes its numbers different from every other project's.*

1.1 A business made of definitions

A health insurance payer collects premium, pays claims, and lives on the difference. Every part of that sentence is a definition someone has to own - which claims, incurred when, against which members, over what period.

That makes it the right final project. The metrics are contractual, the models set payments rather than rankings, an adversary profits from beating the controls, and the whole thing is audited by someone who will ask how a number was produced.

ALL FIVE ROLES, GENUINELY RATHER THAN NOMINALLY

The brief for this project was a single system requiring Data Analyst, Data Engineer, QA/SDET, ML Engineer and AI Engineer work at once - not five components sharing a repository.

A payer delivers that without contrivance: **the analyst needs governed metric definitions, the engineer needs the warehouse and the regression harness that protects them, QA needs a traceability matrix because the domain is regulated, the ML engineer needs calibrated risk models because the output is money, and the AI engineer needs a copilot that cannot invent a definition.**

1.2 What was built and measured

1.1 Measured results across the platform

Layer	Headline result
Semantic layer	one definition per metric, dimensions declared
Metric regression	**one definition change restated 33 of 50 published figures**
Validation	12 requirements, 11 covered - and the gap is REQ-012
Survival analysis	discrete-time hazard 26x more accurate than a churn label
Competing risks	1-KM overstates voluntary lapse by 10.6 points
Upcoding detection	**recall 0.9545 baseline, 0.1818 under the cheapest adaptation**
Risk adjustment	ridge beat gradient boosting; R^2 0.125, calibrated
Hierarchical forecast	top-down 9.55% MAPE beats OLS and MinT reconciliation
Metric copilot	rules 0.875, LLM 0.250, LLM refusal accuracy 0.000

1.3 The result the project exists for

A rules-based detector looks for providers whose billing intensity is anomalous within their own specialty. Against an adversary who does not know it exists, it works well.

1.2 One detector, four scenarios

Scenario	What the fraudster changed	Recall
baseline	nothing - unaware of the detector	**0.9545**
stay_below_threshold	never bills the top code	0.7727
spread_thin	half the ring stops entirely	0.5000
reduced_rate	**upcodes 15% of claims instead of 42%**	**0.1818**

THE CHEAPEST ADAPTATION COST 77.3 POINTS OF RECALL

Abandoning the highest-value code - a real sacrifice - drops recall to 0.77. Shutting down half the operation drops it to 0.50. **Simply upcoding less often drops it to 0.18.**

The fraudster keeps the same scheme, the same codes and the same specialty. They give up roughly two-thirds of the fraudulent volume and become five times harder to detect - which, for anyone doing the arithmetic, is an excellent trade.

A detector evaluated only against a static adversary has not been evaluated. 95.45% is a true number about a scenario that stops being true the moment anyone notices the control exists.

CHAPTER 2

A Metric Is a Contract

> *Why a governed semantic layer exists, what it forbids, and the difference between a query and a definition.*

2.1 The problem it solves

Ask three analysts at a health plan for the loss ratio and you may get three numbers. Not because anyone is wrong - because *loss ratio* is a definition, and definitions drift.

- Does it include pharmacy claims?
- Are claims counted when incurred or when paid?
- Does the denominator include reinsurance premium?
- Are members with zero months in the period in the denominator?

Each answer is defensible. What is not defensible is a company where different reports encode different answers and nobody can tell which.

METRICS-AS-CODE MAKES THE DEFINITION THE ARTEFACT

The semantic layer holds one definition per metric, in version control, with declared dimensions. Consumers request `loss_ratio` sliced by `plan`; they do not write the SQL, so they cannot express a different definition by accident.

A metric stops being a query somebody wrote and becomes a contract the platform enforces. The cost is flexibility - you may only ask what the registry knows - and the benefit is that two reports quoting the same metric are quoting the same thing.

The metric registry — platform/semantic/metrics.py

```
"""
metrics.py
----------
The semantic layer: every business metric defined exactly once, in code.

The problem this solves is the one that actually breaks analytics teams.
"Loss ratio" gets written in a dashboard, a board pack, a regulatory
filing and an actuary's notebook, and the four disagree - usually because
one includes pharmacy, one nets out reinsurance, and nobody wrote down
which. The numbers are not wrong so much as unowned.

Here a metric is a DEFINITION, not a query:

    Metric(name="loss_ratio",
           numerator="SUM(paid_amount)",
           denominator="SUM(premium_collected)",
           grain="member_month",
           filters=["status = 'paid'"])

Queries are COMPILED from a metric plus a set of dimensions, so a
dashboard cannot express "loss ratio, but computed slightly differently".
Two consequences follow, and both are the point:

  1. Every figure on every screen traces to one definition.
  2. A definition CHANGE becomes a reviewable event with a measurable
     blast radius, which is what `integrity/metric_regression.py` exists
     to compute. Without a semantic layer you cannot even ask "which
     historical numbers did this change move?", because there is no
     single thing that changed.

DIMENSIONS are declared too, so a request for a dimension that does not
exist fails loudly at compile time rather than silently returning a
cartesian product.
"""

from dataclasses import dataclass, field


@dataclass(frozen=True)
class Dimension:
    name: str
    sql: str
    description: str


@dataclass(frozen=True)
class Metric:
    """A metric is a ratio (or a plain aggregate when denominator is None),
    plus the grain it is valid at and the filters it always carries."""

    name: str
    label: str
    numerator: str
    denominator: str | None
    grain: str
    description: str = ""
    unit: str = "ratio"
    filters: tuple = ()
    higher_is_better: bool | None = None
    owner: str = "actuarial"
```

2.2 What the registry forbids

3.1 Three invariants the semantic layer enforces

Rule	Prevents
A metric cannot be defined twice	two teams shipping competing definitions of the same name
A query may not reference an undeclared dimension	silently slicing by a column the metric was never validated against
The compiler emits the SQL, not the caller	a consumer joining at the wrong grain and double-counting

THE GRAIN RULE IS THE ONE THAT QUIETLY SAVES YOU

A loss ratio joined to a member-months table at the wrong grain produces a number that is wrong by a factor nobody notices, because it is still in a plausible range. Project 4's grain chapter makes the same argument for a fact table; here it is enforced by construction rather than by an assertion.

It is also what makes the AI copilot in Chapter 12 safe to build: because the model resolves a question to a metric name and a dimension list rather than writing SQL, **it cannot join at the wrong grain even if it wanted to.**

CHAPTER 3

One Definition Change, Thirty-Three Restated Figures

> *Metric regression testing, a blast radius measured rather than estimated, and the one requirement in the traceability matrix with no test behind it.*

3.1 Golden values

The platform snapshots 50 figures across ten metric-and-dimension combinations, and asserts them on every build.

The ten snapshot specifications, expanding to 50 values

```
loss_ratio|total     pmpm|total          member_months|total
loss_ratio|plan      pmpm|region         paid_amount|region
loss_ratio|region    denial_rate|total
loss_ratio|month     denial_rate|plan
```

4.1 The routine regression check

Check	Result
Values snapshotted	50
Moved since the golden file	**0**
Unchanged	50
Verdict	no drift - the warehouse reproduces the golden values

THIS IS A UNIT TEST FOR THE BUSINESS, NOT FOR THE CODE

A software test asserts that a function returns what it should. A metric regression test asserts that *the number the board sees this month is computed the same way it was last month.*

Those are different failures. Code can be refactored correctly and still change a published figure, because the change was in a join condition, a filter, or a definition - and nothing in the test suite was watching the output value.

3.2 The blast radius of a clarification

Now make a change that any analyst would call an improvement: **loss ratio now excludes pharmacy claims.** It is a defensible definition, arguably a more standard one, and it is the kind of thing that gets agreed in a meeting.

4.2 The measured blast radius

Outcome	Count
Figures compared	50
Figures moved	**33**
Materially moved	**33**
Unchanged	17

THIRTY-THREE OF FIFTY PUBLISHED FIGURES WERE SILENTLY RESTATED

Every one of the 33 moves is material. Bronze plan loss ratio goes from 1.0585 to 1.0345; November 2026 from 0.9634 to 0.9404 - moves of around 2.3% relative.

Two point three percent sounds survivable until you notice what 1.0585 means: **a loss ratio above 1.0 is a plan paying out more than it collects.** A change that moves several plans across reporting thresholds, in a regulated filing, is not a clarification - it is a restatement.

Why 17 figures did not move

Only the `loss_ratio` specifications are affected - `pmpm`, `denial_rate`, `member_months` and `paid_amount` use different definitions and are untouched. The 33 that moved are every loss-ratio slice; the 17 that did not are the other metrics.

WHICH IS EXACTLY WHY THE BLAST RADIUS MUST BE COMPUTED, NOT ESTIMATED

Asked in advance how many figures a loss-ratio change would affect, a reasonable engineer would say 'the loss ratio ones'. That is correct and useless - the actionable number is **33 specific published values, named**, so each can be checked, reissued or footnoted.

This is the same lineage argument as Project 13's blast radius over downstream consumers, applied one level up: not *which systems read this table*, but *which numbers already in circulation just became wrong*.

3.3 The traceability matrix, and its one gap

Regulated software carries a requirements traceability matrix: every requirement maps to a specification and to the tests that verify it.

4.3 The traceability matrix

Metric	Value
Requirements	12
Covered	11
Coverage	**91.7%**
Gaps	1

The single uncovered requirement

```
REQ-012  "A metric definition change publishes its blast radius"
         reason: no test declared
```

THE ONE UNTESTED REQUIREMENT IS THE CAPABILITY THAT PRODUCED THIS CHAPTER'S RESULT

REQ-012 is the blast-radius publication - the exact machinery that just discovered 33 restated figures. It works. It has no test asserting that it works.

That is precisely the failure mode a traceability matrix exists to surface, and it surfaced it. A capability that is exercised manually and produces good output is indistinguishable, to a regulator, from one that happens to work today - and 91.7% coverage with the gap *named* is far more useful than an unexamined 100%.

The traceability matrix builder qa/validation.py

```
"""
validation.py
---------------
The QA discipline a regulated payer actually has to satisfy, plus the one
measurement that tells you whether the test suite is worth anything.

Three parts:

  TRACEABILITY   every requirement maps to a specification, a test, and
                 the evidence that test produced. An auditor's first
                 question is not "do you have tests" but "show me the test
                 that proves requirement REQ-004 and the run that passed
                 it". A requirement with no test is reported as a gap
                 rather than quietly omitted.

  AUDIT TRAIL    an append-only, hash-chained record of every automated
                 decision. Chaining matters: it makes silent edits
                 detectable, which is the difference between a log and an
                 audit trail. The chain is deliberately tampered with at
                 the end to prove the verifier notices.

  INJECTED BUGS  ten real defects planted in the analytics code, with the
                 suite run against each. A green suite proves the tests
                 pass; it says nothing about whether they would fail if
                 the code were wrong, and that is the only property worth
                 measuring.

Output: reports/validation.json
"""

import hashlib
import json
import shutil
import subprocess
import sys
import tempfile
from datetime import datetime, timezone
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
REPORTS = ROOT / "reports"
REPORTS.mkdir(exist_ok=True)
PY = sys.executable

# ============================================================================
#  Requirements -> specification -> test
# ============================================================================
REQUIREMENTS = [
    ("REQ-001", "Every business metric has exactly one definition",
     "platform/semantic/metrics.py::Registry.add_metric",
     ["test_a_metric_cannot_be_defined_twice"]),
    ("REQ-002", "A query may not reference an undeclared dimension",
     "platform/semantic/metrics.py::Metric.compile",
     ["test_unknown_dimension_is_rejected"]),
    ("REQ-003", "A query may not reference an undefined metric",
```

It is also a pleasing demonstration that the matrix is real rather than decorative. A traceability matrix generated to look complete would have reported 12 of 12.

The metric regression harness — integrity/metric_regression.py

```
"""
metric_regression.py
----------------------
The test suite for the numbers themselves.

Software has regression tests. Metrics almost never do, and the failure
mode is worse: when a metric definition changes, every historical figure
computed from it changes too, silently and retroactively. Last quarter's
loss ratio in the board pack no longer matches the warehouse, nobody
notices for a month, and the reconciliation costs more than the original
work.

A semantic layer makes this testable, because there is exactly one thing
that changed. Three checks run here:

  GOLDEN VALUES     every metric x dimension combination is snapshotted.
                    A later run diffs against the snapshot, so an
                    unintended movement fails loudly.

  BLAST RADIUS      a REAL definition change is applied - including
                    pharmacy in the loss ratio, which is a live argument
                    at every payer - and the report states exactly which
                    historical figures moved, by how much, and whether any
                    crossed a threshold that matters.

  INVARIANTS        properties that must hold regardless of definition:
                    ratios in range, numerator and denominator reconciling
                    to the base table, dimensional slices summing to the
                    total. These catch a broken definition that a golden
                    file cannot, because a golden file is only ever as
                    correct as the day it was written.

Output: reports/metric_regression.json, integrity/golden_metrics.json
"""

import json
import sys
from pathlib import Path

import duckdb

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT / "platform" / "semantic"))
from metrics import REGISTRY, Metric, Registry, Dimension, compile_query  # noqa: E402

DB_PATH = ROOT / "database" / "aegis.duckdb"
REPORTS = ROOT / "reports"
REPORTS.mkdir(exist_ok=True)
GOLDEN = Path(__file__).resolve().parent / "golden_metrics.json"

# Combinations that get snapshotted. Deliberately includes the headline
# (no dimensions) and the slices a board pack actually shows.
SNAPSHOT_SPECS = [
    ("loss_ratio", []), ("loss_ratio", ["plan"]), ("loss_ratio", ["region"]),
    ("loss_ratio", ["month"]),
```

CHAPTER 4

A Guided Tour of the Codebase

> *Every module that matters, in the order the pipeline runs them, with its purpose and public surface. This chapter is generated from the repository, so it cannot describe a function that no longer exists.*

4.1 How to read this tour

Read the generator first - every result in this book is scored against mechanisms it injects. Then the semantic layer and the regression harness, which are what make the analytics trustworthy rather than merely correct. The survival, risk and forecasting chapters are independent and can be read in any order.

4.2 platform/ingest/generate_payer_data.py

Why it exists. The seeded payer book: members, enrolment spells, claims, providers, and the injected mechanisms - the survival coefficients, the upcoding ring and the competing exit causes - that everything downstream is graded against.

What the module says about itself. Generates the Aegis book of business, with every injected mechanism recorded so downstream analysis can be scored rather than admired.

Public functions in platform/ingest/generate_payer_data.py

Function	What it does
generate_members(rng)	—
simulate_lapse(members, rng)	Month-by-month survival simulation with a known hazard.
generate_providers(rng)	—
generate_claims(members, spans, providers, rng)	One row per claim line, only within a member's enrolled months.
main()	—

Opening of the module — platform/ingest/generate_payer_data.py

```
"""
generate_payer_data.py
-------------------------
Generates the Aegis book of business, with every injected mechanism
recorded so downstream analysis can be scored rather than admired.

The design point that matters most is CENSORING. Members are simulated
month by month with a real hazard of lapsing, and the observation window
closes at OBSERVATION_END_MONTH. A member still enrolled at that point has
not "not churned" - their lapse time is simply unknown and greater than
their observed tenure. Recording `event_observed = 0` for them is what
makes survival analysis possible and what makes the naive
churned-yes/no control arm measurably wrong.

Outputs:
    data/members.csv              one row per member (demographics, plan)
    data/enrollment_spans.csv     tenure + event/censoring flag
    data/providers.csv
    data/claim_lines.csv          the fact table
    data/clinical_notes.csv       free text for the AI layer
    data/ground_truth_payer.json  the answer key
"""

import json
import sys
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parent.parent.parent
sys.path.insert(0, str(Path(__file__).resolve().parent))
from domain import (  # noqa: E402
    BASELINE_MONTHLY_HAZARD, CHANNELS, CLAIM_STATUS, DENIAL_REASONS,
    EM_ALLOWED_AMOUNT, EM_CODES, EM_INTENSITY, EM_MINUTES,
    FRAUD_RING_SPECIALTIES, LAPSE_CAUSE_SHARES, LAPSE_COEFFS,
    LEVEL_SHIFT_MONTH, LEVEL_SHIFT_REGION, LEVEL_SHIFT_SIZE, N_MEMBERS,
    N_MONTHS, N_FRAUD_PROVIDERS, N_PROVIDERS, NULL_PROXY_NAME,
    NULL_PROXY_TRUE_EFFECT, OBSERVATION_END_MONTH, PLACE_OF_SERVICE, PLANS,
    PRODUCTS, REGION_MONTHLY_TREND, REGIONS, RISK_COEFFS, SEASONAL_AMPLITUDE,
    SEED_CLAIMS, SEED_FRAUD, SEED_LAPSE, SEED_MEMBERS, SEED_NOTES,
    SEED_PROVIDERS, SERVICE_CATEGORIES, SPECIALTIES, UPCODE_SHIFT, COST_SCALE,
    month_label,
)

DATA = ROOT / "data"
DATA.mkdir(exist_ok=True)

PLAN_PREMIUM = {"Bronze": 310.0, "Silver": 430.0, "Gold": 585.0, "Platinum": 760.0}
PLAN_ACTUARIAL_VALUE = {"Bronze": 0.60, "Silver": 0.70, "Gold": 0.80, "Platinum": 0.90}


# ---------------------------------------------------------------------
#  Members
# ---------------------------------------------------------------------
def generate_members(rng):
    n = N_MEMBERS
    plan = rng.choice(PLANS, size=n, p=[0.28, 0.37, 0.24, 0.11])
    product = rng.choice(PRODUCTS, size=n, p=[0.44, 0.41, 0.15])
    region = rng.choice(REGIONS, size=n, p=[0.24, 0.21, 0.19, 0.20, 0.16])
    channel = rng.choice(CHANNELS, size=n, p=[0.52, 0.21, 0.19, 0.08])

    age = np.clip(rng.normal(44, 16, n), 18, 89).astype(int)
    income = np.clip(rng.lognormal(10.9, 0.52, n), 14_000, 400_000)
    chronic = rng.poisson(np.clip(0.35 + (age - 40) * 0.021, 0.05, None), n)

    premium = np.array([PLAN_PREMIUM[p] for p in plan])
    premium_burden = np.clip(premium * 12 / income, 0.01, 0.60)

    # A variable with ZERO true effect on cost that correlates with region,
    # so the fairness audit has something real to find nothing in.
    region_base = {r: v for r, v in zip(sorted(REGIONS),
```

4.3 platform/warehouse/build_warehouse.py

Why it exists. Builds the warehouse the semantic layer sits on.

What the module says about itself. DuckDB warehouse for Aegis, with the conformed member-month grain the semantic layer depends on.

Public functions in platform/warehouse/build_warehouse.py

Function	What it does
build(con)	—
main()	—

Opening of the module — platform/warehouse/build_warehouse.py

```
"""
build_warehouse.py
--------------------
DuckDB warehouse for Aegis, with the conformed member-month grain the
semantic layer depends on.

The grain declaration is the load-bearing part. `vw_member_month` is
exactly one row per member per enrolled month, carrying that month's
premium and that month's claims. Every ratio metric is defined against it,
which prevents the most common and least visible error in payer
analytics: a numerator aggregated at claim grain divided by a denominator
aggregated at member grain. That produces a loss ratio wrong by whatever
the average claims-per-member happens to be, and it looks entirely
plausible.

Loads are CREATE OR REPLACE and idempotency is asserted rather than
assumed - running the build twice must not change a single figure.

Output: database/aegis.duckdb, reports/warehouse_summary.json
"""

import json
import sys
from pathlib import Path

import duckdb

ROOT = Path(__file__).resolve().parent.parent.parent
sys.path.insert(0, str(ROOT / "platform" / "ingest"))
from domain import N_MONTHS  # noqa: E402

DATA = ROOT / "data"
DB_DIR = ROOT / "database"
DB_DIR.mkdir(exist_ok=True)
DB_PATH = DB_DIR / "aegis.duckdb"
REPORTS = ROOT / "reports"
REPORTS.mkdir(exist_ok=True)

# One row per WHAT? Written down so it is reviewable.
GRAIN = {
    "dim_member": "one row per member",
    "dim_provider": "one row per provider (NPI)",
    "fct_enrollment_span": "one row per member (their single enrolment span)",
    "fct_claim_line": "one row per adjudicated claim LINE",
    "vw_member_month": "one row per member per enrolled month - the conformed "
                       "grain every ratio metric is defined against",
    "vw_provider_coding": "one row per provider per month, professional claims only",
}


def build(con):
    con.execute(f"""
        CREATE OR REPLACE TABLE dim_member AS
        SELECT * FROM read_csv_auto('{(DATA / "members.csv").as_posix()}');

        CREATE OR REPLACE TABLE dim_provider AS
        SELECT * FROM read_csv_auto('{(DATA / "providers.csv").as_posix()}');

        CREATE OR REPLACE TABLE fct_enrollment_span AS
        SELECT * FROM read_csv_auto('{(DATA / "enrollment_spans.csv").as_posix()}');

        CREATE OR REPLACE TABLE fct_claim_line AS
        SELECT * FROM read_csv_auto(
            '{(DATA / "claim_lines.csv").as_posix()}',
            types={{'em_code': 'VARCHAR'}});
    """)

    # A month spine, so a member-month with ZERO claims still exists. This
    # is the difference between "average cost of members who claimed" and
    # PMPM, and conflating them overstates cost by roughly the share of
    # members who used nothing that month.
    con.execute(f"""
```

4.4 platform/semantic/metrics.py

Why it exists. The metric registry: one definition per metric, declared dimensions, and the compiler that emits SQL so consumers never write it.

What the module says about itself. The semantic layer: every business metric defined exactly once, in code.

Classes in platform/semantic/metrics.py

Class	Purpose	Key methods
Dimension	—	—
Metric	A metric is a ratio (or a plain aggregate when denominator is None),	compile
Registry	—	add_metric, add_dimension, get

Public functions in platform/semantic/metrics.py

Function	What it does
compile_query(metric_name, dimensions, where, registry)	—
metric_catalog(registry)	Machine-readable catalogue. The AI layer reads THIS rather than the

Opening of the module — platform/semantic/metrics.py

```
"""
metrics.py
----------
The semantic layer: every business metric defined exactly once, in code.

The problem this solves is the one that actually breaks analytics teams.
"Loss ratio" gets written in a dashboard, a board pack, a regulatory
filing and an actuary's notebook, and the four disagree - usually because
one includes pharmacy, one nets out reinsurance, and nobody wrote down
which. The numbers are not wrong so much as unowned.

Here a metric is a DEFINITION, not a query:

    Metric(name="loss_ratio",
           numerator="SUM(paid_amount)",
           denominator="SUM(premium_collected)",
           grain="member_month",
           filters=["status = 'paid'"])

Queries are COMPILED from a metric plus a set of dimensions, so a
dashboard cannot express "loss ratio, but computed slightly differently".
Two consequences follow, and both are the point:

  1. Every figure on every screen traces to one definition.
  2. A definition CHANGE becomes a reviewable event with a measurable
     blast radius, which is what `integrity/metric_regression.py` exists
     to compute. Without a semantic layer you cannot even ask "which
     historical numbers did this change move?", because there is no
     single thing that changed.

DIMENSIONS are declared too, so a request for a dimension that does not
exist fails loudly at compile time rather than silently returning a
cartesian product.
"""

from dataclasses import dataclass, field


@dataclass(frozen=True)
class Dimension:
    name: str
    sql: str
    description: str


@dataclass(frozen=True)
class Metric:
    """A metric is a ratio (or a plain aggregate when denominator is None),
    plus the grain it is valid at and the filters it always carries."""

    name: str
    label: str
    numerator: str
    denominator: str | None
    grain: str
    description: str = ""
    unit: str = "ratio"
    filters: tuple = ()
    higher_is_better: bool | None = None
    owner: str = "actuarial"

    def compile(self, dimensions: list[str], where: list[str] | None = None,
                registry=None) -> str:
        reg = registry or REGISTRY
        unknown = [d for d in dimensions if d not in reg.dimensions]
        if unknown:
            raise ValueError(
                f"unknown dimension(s) {unknown} for metric '{self.name}'; "
                f"declared dimensions are {sorted(reg.dimensions)}")

        selects = [f"{reg.dimensions[d].sql} AS {d}" for d in dimensions]
        conds = list(self.filters) + list(where or [])
```

4.5 integrity/metric_regression.py

Why it exists. Golden-value snapshots and the blast-radius computation that found 33 restated figures.

What the module says about itself. The test suite for the numbers themselves.

Public functions in integrity/metric_regression.py

Function	What it does
snapshot(con, registry)	—
diff(before, after)	—
build_changed_registry()	The proposed change: loss ratio now EXCLUDES pharmacy.
invariants(con)	Properties that must hold whatever the definitions say.
main()	—

Opening of the module — integrity/metric_regression.py

```
"""
metric_regression.py
----------------------
The test suite for the numbers themselves.

Software has regression tests. Metrics almost never do, and the failure
mode is worse: when a metric definition changes, every historical figure
computed from it changes too, silently and retroactively. Last quarter's
loss ratio in the board pack no longer matches the warehouse, nobody
notices for a month, and the reconciliation costs more than the original
work.

A semantic layer makes this testable, because there is exactly one thing
that changed. Three checks run here:

  GOLDEN VALUES     every metric x dimension combination is snapshotted.
                    A later run diffs against the snapshot, so an
                    unintended movement fails loudly.

  BLAST RADIUS      a REAL definition change is applied - including
                    pharmacy in the loss ratio, which is a live argument
                    at every payer - and the report states exactly which
                    historical figures moved, by how much, and whether any
                    crossed a threshold that matters.

  INVARIANTS        properties that must hold regardless of definition:
                    ratios in range, numerator and denominator reconciling
                    to the base table, dimensional slices summing to the
                    total. These catch a broken definition that a golden
                    file cannot, because a golden file is only ever as
                    correct as the day it was written.

Output: reports/metric_regression.json, integrity/golden_metrics.json
"""

import json
import sys
from pathlib import Path

import duckdb

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT / "platform" / "semantic"))
from metrics import REGISTRY, Metric, Registry, Dimension, compile_query  # noqa: E402

DB_PATH = ROOT / "database" / "aegis.duckdb"
REPORTS = ROOT / "reports"
REPORTS.mkdir(exist_ok=True)
GOLDEN = Path(__file__).resolve().parent / "golden_metrics.json"

# Combinations that get snapshotted. Deliberately includes the headline
# (no dimensions) and the slices a board pack actually shows.
SNAPSHOT_SPECS = [
    ("loss_ratio", []), ("loss_ratio", ["plan"]), ("loss_ratio", ["region"]),
    ("loss_ratio", ["month"]),
    ("pmpm", []), ("pmpm", ["region"]),
    ("denial_rate", []), ("denial_rate", ["plan"]),
    ("member_months", []), ("paid_amount", ["region"]),
]

# A movement larger than this in a headline ratio is a restatement, not a
# rounding difference. Set before running, not after seeing the answer.
MATERIALITY_RATIO = 0.005          # half a percentage point
MATERIALITY_RELATIVE = 0.01        # or 1% relative, whichever binds first


def snapshot(con, registry=None):
    out = {}
    for metric, dims in SNAPSHOT_SPECS:
        key = f"{metric}|{'+'.join(dims) if dims else 'total'}"
        rows = con.execute(compile_query(metric, dims, registry=registry)).fetchall()
        if dims:
```

4.6 integrity/upcoding_detection.py

Why it exists. Three signals, the volume floor, and the red-team scenarios where the detector's recall collapses.

What the module says about itself. Finding providers who bill a higher-intensity E/M code than the encounter justifies, and then finding out how much of that detection survives an adversary who knows the rules.

Public functions in integrity/upcoding_detection.py

Function	What it does
load()	—
provider_features(prof, providers)	—
peer_scores(g)	z-scores WITHIN specialty. Peer choice is the whole ballgame: judged
apply_rules(g)	—
evaluate(scored, label)	—
simulate_evasion(prof, providers, strategy, rng)	Rebuild the claim stream with an adapted adversary.
main()	—

Opening of the module — integrity/upcoding_detection.py

```
"""
upcoding_detection.py
-----------------------
Finding providers who bill a higher-intensity E/M code than the encounter
justifies, and then finding out how much of that detection survives an
adversary who knows the rules.

Detection uses BEHAVIOUR only. `is_fraud` exists in the provider table and
is used solely to score - never as a feature, never as a filter. The same
discipline applies to `true_em_code`, which lives in a separate scoring
file so it cannot be joined in by accident.

Three signals, each with an independent rationale:

  PEER INTENSITY   a provider's mean E/M level against peers in the SAME
                   specialty. Specialty matters: an oncologist legitimately
                   bills higher than a family doctor, and comparing across
                   specialties would flag the entire oncology department.
  DISTRIBUTION     the share of a provider's claims at the top two levels.
                   Catches a provider whose mean looks normal because they
                   balance high codes with low ones.
  IMPOSSIBLE DAY   billed encounter minutes exceeding a plausible working
                   day. A hard rule with no statistics in it, included
                   because it is the one an investigator can defend without
                   explaining a z-score to a lawyer.

THE RED TEAM
------------
A detector's recall against a static adversary is close to meaningless,
because the adversary is not static. Once thresholds are known, upcoding
adapts: bill just under the flagging line, spread the behaviour across
more encounters, or shift only within the codes where peers are diffuse.
Each evasion is simulated and recall is re-measured, so the reported
strength is strength under attack rather than strength on the training
distribution.

Output: reports/upcoding_detection.json
"""

import json
import sys
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT / "platform" / "ingest"))
from domain import (  # noqa: E402
    EM_CODES, EM_INTENSITY, EM_MINUTES, IMPOSSIBLE_DAY_MINUTES, UPCODE_SHIFT,
)

DATA = ROOT / "data"
REPORTS = ROOT / "reports"
REPORTS.mkdir(exist_ok=True)

MIN_CLAIMS_FOR_SCORING = 40      # below this a provider's mean is noise
FLAG_Z = 2.5                     # peer-relative z-score threshold
TOP_CODE_SHARE_Z = 2.5


def load():
    claims = pd.read_csv(DATA / "claim_lines.csv", dtype={"em_code": "string"})
    providers = pd.read_csv(DATA / "providers.csv")
    prof = claims[claims.service_category == "professional"].copy()
    prof = prof[prof.em_code.notna()]
    prof["intensity"] = prof.em_code.map(EM_INTENSITY)
    prof["minutes"] = prof.em_code.map(EM_MINUTES)
    return prof, providers


def provider_features(prof, providers):
```

4.7 analytics/survival/member_survival.py

Why it exists. Kaplan-Meier with Greenwood intervals, the log-rank test, three estimators graded against injected coefficients, and the competing-risks CIF.

What the module says about itself. Time-to-lapse analysis on enrolment spans where 35.6% of members have NOT lapsed by the time we look.

Public functions in analytics/survival/member_survival.py

Function	What it does
load()	—
kaplan_meier(durations, events)	Non-parametric survival, with Greenwood variance.
median_survival(km)	—
logrank(d1, e1, d2, e2)	Log-rank test between two groups. Compares observed events to those
cox_ph(X, durations, events)	Breslow partial likelihood, vectorised.
person_month(d)	Explode spans into one row per member-month at risk, vectorised.
main()	—

Opening of the module — analytics/survival/member_survival.py

```
"""
member_survival.py
--------------------
Time-to-Lapse analysis on enrolment spans where 35.6% of members have NOT
lapsed by the time we look.

Those members are the whole point. They are RIGHT-CENSORED: their lapse
time is unknown and greater than their observed tenure. The instinctive
move - label them "did not churn" and fit a classifier - throws away the
distinction between "stayed 22 months and is still here" and "stayed 2
months and is still here", and treats the second as evidence of loyalty.

Four estimators are fitted on identical data and scored against the hazard
the generator injected:

  NAIVE LOGISTIC     churned yes/no, ignoring tenure entirely. The control
                     arm, and the thing most teams actually ship.
  KAPLAN-MEIER       non-parametric survival by cohort, with Greenwood
                     confidence bands and at-risk counts.
  COX PH             partial likelihood (Breslow ties), hand-implemented.
                     The right family; a continuous-time approximation to
                     a process that is actually discrete.
  DISCRETE-TIME      person-month logistic hazard. This one MATCHES the
  HAZARD             data-generating process exactly, and is included
                     because "Cox is the survival model" is a habit rather
                     than a decision - when data arrive in monthly
                     buckets, the discrete-time model is the correct one.

Everything is implemented directly rather than through lifelines: the
partial likelihood is fifteen lines, and writing it is the difference
between using survival analysis and understanding it. It also keeps the
project runnable with no extra dependency.

Output: reports/survival.json
"""

import json
import sys
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.optimize import minimize
from sklearn.linear_model import LogisticRegression

ROOT = Path(__file__).resolve().parent.parent.parent
sys.path.insert(0, str(ROOT / "platform" / "ingest"))
from domain import LAPSE_COEFFS, N_MONTHS  # noqa: E402

DATA = ROOT / "data"
REPORTS = ROOT / "reports"
REPORTS.mkdir(exist_ok=True)

# The covariates the generator actually used. Named here so the scoring is
# a like-for-like comparison rather than a fishing expedition.
COVARIATES = ["premium_burden_x10", "had_denial", "utilisation_low",
              "channel_individual"]


def load():
    spans = pd.read_csv(DATA / "enrollment_spans.csv")
    members = pd.read_csv(DATA / "members.csv")
    d = spans.merge(members[["member_id", "premium_burden", "channel", "region",
                             "plan", "age_band"]], on="member_id")
    # premium_burden enters the generator as burden*10; matching the scale
    # here means the recovered coefficient is directly comparable
    d["premium_burden_x10"] = d.premium_burden * 10
    d["channel_individual"] = (d.channel == "individual").astype(int)
    return d


# ============================================================================
```

4.8 analytics/risk/risk_adjustment.py

Why it exists. Time-split cost prediction with calibration by decile - and the stated reason the split is temporal.

What the module says about itself. Predicting member cost, and being judged on whether the predictions are RIGHT rather than merely well-ordered.

Public functions in analytics/risk/risk_adjustment.py

Function	What it does
build_dataset(con)	One row per member: features from the first year, target from the
calibration(y_true, y_pred, n_bins)	Predicted vs actual by decile of prediction, plus the slope of a
evaluate(name, y_true, y_pred)	—
main()	—

Opening of the module — analytics/risk/risk_adjustment.py

```
"""
risk_adjustment.py
--------------------
Predicting member cost, and being judged on whether the predictions are
RIGHT rather than merely well-ordered.

Risk adjustment is not a ranking problem. The output sets what a plan is
paid for a member, so a model that orders members perfectly but is
systematically 30% low prices every contract wrong and loses money on all
of them simultaneously. Discrimination (AUC, Gini) cannot see that;
calibration can, and it is the metric this module leads with.

Evaluated here:

  DISCRIMINATION   Spearman rank correlation and decile lift - can the
                   model tell an expensive member from a cheap one?
  CALIBRATION      predicted vs actual by decile, calibration slope and
                   intercept, and the payment error that follows from
                   miscalibration. A slope below 1 means the model is
                   compressed toward the mean and systematically
                   underpays the sick and overpays the healthy.
  FAIRNESS         a variable with ZERO true effect on cost is included in
                   the data and correlates with region. Project 11 found
                   such a proxy did not drive disparate outcomes once real
                   signal was present; this re-tests that claim on a
                   different population rather than assuming it repeats.

The split is TIME-BASED. A random split lets a member's own later months
inform their earlier ones, and in a book where cost is highly
autocorrelated that inflates every metric.

Output: reports/risk_adjustment.json
"""

import json
import sys
from pathlib import Path

import duckdb
import numpy as np
from scipy.stats import spearmanr
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.linear_model import Ridge

ROOT = Path(__file__).resolve().parent.parent.parent
sys.path.insert(0, str(ROOT / "platform" / "ingest"))
from domain import NULL_PROXY_NAME, NULL_PROXY_TRUE_EFFECT, RISK_COEFFS  # noqa: E402

DB_PATH = ROOT / "database" / "aegis.duckdb"
DATA = ROOT / "data"
REPORTS = ROOT / "reports"
REPORTS.mkdir(exist_ok=True)

TRAIN_MONTHS = 12          # predict months 12-23 from months 0-11
FEATURES = ["age_band", "chronic_count", "prior_cost_log", "prior_inpatient",
            "prior_lines", NULL_PROXY_NAME]


def build_dataset(con):
    """One row per member: features from the first year, target from the
    second. Point-in-time by construction - nothing from the target window
    can enter a feature."""
    return con.execute(f"""
        WITH prior AS (
            SELECT member_id,
                   SUM(paid_amount)                        AS prior_cost,
                   SUM(total_lines)                        AS prior_lines,
                   COUNT(*)                                AS prior_months
            FROM vw_member_month WHERE month_index < {TRAIN_MONTHS}
            GROUP BY 1
        ),
        prior_ip AS (
```

4.9 analytics/forecasting/hierarchical_forecast.py

Why it exists. Four reconciliation approaches over a 24-series hierarchy, scored on a 6-month holdout.

What the module says about itself. Forecasting PMPM across a hierarchy whose levels must sum.

Public functions in analytics/forecasting/hierarchical_forecast.py

Function	What it does
load_panel(con)	PMPM per region x product per month, plus the aggregates.
summing_matrix(keys)	S maps leaf series to every series in the hierarchy.
base_forecast(series, h, window, phi)	Damped-trend forecast with an additive seasonal term.
reconcile_ols(S, base)	Project base forecasts onto the coherent subspace.
reconcile_mint_diag(S, base, residual_var)	MinT with a diagonal error covariance - scale-free weighting, so a
main()	—

Opening of the module — analytics/forecasting/hierarchical_forecast.py

```
"""
hierarchical_forecast.py
--------------------------
Forecasting PMPM across a hierarchy whose levels must sum.

The planning hierarchy is region x product, aggregating to region, to
product, and to the total. Forecasts are produced independently at each
level because different people own them - and independent forecasts do
not add up. The regional numbers will not sum to the total, and both
appear in the same board pack.

That inconsistency is not cosmetic. It means at least one of the numbers
someone is being held to is not the number the plan was built on.

Reconciliation fixes it, and the interesting question is whether it costs
anything:

  BASE          independent forecasts per series. Incoherent by
                construction; the incoherence is measured.
  BOTTOM-UP     forecast the leaves, sum upward. Always coherent, and
                throws away whatever signal the aggregate series had -
                aggregates are less noisy than their parts.
  TOP-DOWN      forecast the total, split by historical share. Coherent,
                stable at the top, and unable to represent a leaf
                diverging from its siblings.
  OLS / MinT    project the base forecasts onto the coherent subspace,
                using all levels at once. Should beat both, and this file
                measures whether it actually does rather than assuming it.

WHY PMPM AND NOT TOTAL COST
--------------------------
Total claims cost confounds price with membership. This book shrinks as
members lapse, so total cost can fall while cost per member rises - and a
forecaster trained on totals would project the shrinkage forward as if it
were a cost improvement. PMPM is what actuaries forecast, and it is the
target here for that reason.

Output: reports/hierarchical_forecast.json
"""

import json
import sys
from pathlib import Path

import duckdb
import numpy as np

ROOT = Path(__file__).resolve().parent.parent.parent
sys.path.insert(0, str(ROOT / "platform" / "ingest"))
from domain import (  # noqa: E402
    LEVEL_SHIFT_MONTH, LEVEL_SHIFT_REGION, LEVEL_SHIFT_SIZE, N_MONTHS,
    REGION_MONTHLY_TREND, SEASONAL_AMPLITUDE,
)

DB_PATH = ROOT / "database" / "aegis.duckdb"
REPORTS = ROOT / "reports"
REPORTS.mkdir(exist_ok=True)

HOLDOUT = 6                # forecast the final 6 months
SEASON = 12


def load_panel(con):
    """PMPM per region x product per month, plus the aggregates."""
    rows = con.execute("""
        SELECT region, product, month_index,
               SUM(paid_amount) / COUNT(*) AS pmpm
        FROM vw_member_month
        GROUP BY 1, 2, 3 ORDER BY 1, 2, 3
    """).fetchall()
    leaves, months = {}, set()
    for r, p, m, v in rows:
```

4.10 ai/metric_copilot.py

Why it exists. The copilot that resolves questions to a governed metric and dimensions and never writes SQL.

What the module says about itself. Natural-language questions answered through the SEMANTIC LAYER, not through generated SQL.

Public functions in ai/metric_copilot.py

Function	What it does
call_ollama(prompt, seed)	—
parse(text)	Strict parse against the closed vocabulary.
rules_baseline(question)	Keyword matching. The control arm - if the model cannot beat this,
score(pred_metric, pred_dims, want_metric, want_dims)	—
main()	—

Opening of the module — ai/metric_copilot.py

```
"""
metric_copilot.py
-------------------
Natural-language questions answered through the SEMANTIC LAYER, not
through generated SQL.

The usual text-to-SQL design hands a model the schema and asks for a
query. On a payer's warehouse that is close to the worst available idea:
the model can invent a definition of loss ratio, join at the wrong grain,
or read a column it has no business reading, and the answer comes back
formatted confidently either way.

Here the model does something much smaller and much safer. It maps the
question onto a CLOSED VOCABULARY - one of seven governed metrics and a
handful of declared dimensions - and the platform compiles the query
itself. The model cannot express a wrong definition because it never
writes a definition. The worst it can do is pick the wrong metric, and
that is both detectable and recoverable.

Three things are measured:

  RESOLUTION ACCURACY   does it pick the right metric and dimensions,
                        scored against an answer key of questions?
  RULES BASELINE        keyword matching, which is what this would be
                        without a model. If the model cannot beat it, the
                        model is decoration.
  REFUSAL               questions the vocabulary genuinely cannot answer
                        must be REFUSED, not answered with the nearest
                        available metric. An analytics assistant that
                        always produces a number is more dangerous than
                        one that sometimes says no.

Runs on local Ollama qwen2.5:0.5b - free, offline, no expiry.

Output: reports/metric_copilot.json
"""

import json
import re
import sys
import time
import urllib.error
import urllib.request
from pathlib import Path

import duckdb

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT / "platform" / "semantic"))
from metrics import REGISTRY, compile_query, metric_catalog  # noqa: E402

DB_PATH = ROOT / "database" / "aegis.duckdb"
REPORTS = ROOT / "reports"
REPORTS.mkdir(exist_ok=True)

OLLAMA = "http://localhost:11434/api/generate"
MODEL = "qwen2.5:0.5b"
TIMEOUT = 120

METRIC_NAMES = sorted(REGISTRY.metrics)
DIM_NAMES = sorted(REGISTRY.dimensions)

# The answer key. Written before running anything, and deliberately
# includes questions the vocabulary CANNOT serve.
QUESTIONS = [
    ("What is our loss ratio?", "loss_ratio", []),
    ("Show loss ratio by plan", "loss_ratio", ["plan"]),
    ("How does the loss ratio break down by region?", "loss_ratio", ["region"]),
    ("What is PMPM?", "pmpm", []),
    ("Show me PMPM by region", "pmpm", ["region"]),
    ("What is the denial rate?", "denial_rate", []),
    ("Denial rate by plan please", "denial_rate", ["plan"]),
```

4.11 qa/validation.py

Why it exists. The requirements traceability matrix and its one honest gap.

What the module says about itself. The QA discipline a regulated payer actually has to satisfy, plus the one measurement that tells you whether the test suite is worth anything.

Classes in qa/validation.py

Class	Purpose	Key methods
AuditTrail	Append-only, hash-chained.	append, verify

Public functions in qa/validation.py

Function	What it does
run_suite(cwd, timeout)	—
collect_test_names()	—
main()	—

Opening of the module — qa/validation.py

```
"""
validation.py
---------------
The QA discipline a regulated payer actually has to satisfy, plus the one
measurement that tells you whether the test suite is worth anything.

Three parts:

  TRACEABILITY   every requirement maps to a specification, a test, and
                 the evidence that test produced. An auditor's first
                 question is not "do you have tests" but "show me the test
                 that proves requirement REQ-004 and the run that passed
                 it". A requirement with no test is reported as a gap
                 rather than quietly omitted.

  AUDIT TRAIL    an append-only, hash-chained record of every automated
                 decision. Chaining matters: it makes silent edits
                 detectable, which is the difference between a log and an
                 audit trail. The chain is deliberately tampered with at
                 the end to prove the verifier notices.

  INJECTED BUGS  ten real defects planted in the analytics code, with the
                 suite run against each. A green suite proves the tests
                 pass; it says nothing about whether they would fail if
                 the code were wrong, and that is the only property worth
                 measuring.

Output: reports/validation.json
"""

import hashlib
import json
import shutil
import subprocess
import sys
import tempfile
from datetime import datetime, timezone
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
REPORTS = ROOT / "reports"
REPORTS.mkdir(exist_ok=True)
PY = sys.executable

# ---------------------------------------------------------------------
#  Requirements -> specification -> test
# ---------------------------------------------------------------------
REQUIREMENTS = [
    ("REQ-001", "Every business metric has exactly one definition",
     "platform/semantic/metrics.py::Registry.add_metric",
     ["test_a_metric_cannot_be_defined_twice"]),
    ("REQ-002", "A query may not reference an undeclared dimension",
     "platform/semantic/metrics.py::Metric.compile",
     ["test_unknown_dimension_is_rejected"]),
    ("REQ-003", "A query may not reference an undefined metric",
     "platform/semantic/metrics.py::Registry.get",
     ["test_unknown_metric_is_rejected"]),
    ("REQ-004", "Ratio metrics equal numerator divided by denominator",
     "platform/semantic/metrics.py::Metric.compile",
     ["test_ratio_equals_numerator_over_denominator"]),
    ("REQ-005", "Dimensional slices reconcile to the reported total",
     "platform/warehouse/build_warehouse.py::vw_member_month",
     ["test_dimensional_slices_reconcile_on_the_numerator"]),
    ("REQ-006", "Member-months equal the sum of enrolment durations (no fan-out)",
     "platform/warehouse/build_warehouse.py::vw_member_month",
     ["test_member_months_equal_the_sum_of_enrolment_durations"]),
    ("REQ-007", "Member-months with zero claims remain in the denominator",
     "platform/warehouse/build_warehouse.py::dim_month spine",
     ["test_zero_claim_member_months_stay_in_the_denominator"]),
    ("REQ-008", "Censored members do not reduce estimated survival",
     "analytics/survival/member_survival.py::kaplan_meier",
     ["test_censored_observations_do_not_drop_survival",
```

4.12 tests/test_semantic_and_analytics.py

Why it exists. The suite the traceability matrix maps requirements onto.

What the module says about itself. Tests for the properties that make Aegis's numbers trustworthy.

Public functions in tests/test_semantic_and_analytics.py

Function	What it does
con()	—
test_unknown_dimension_is_rejected()	Governance must fail at compile time, not return a wrong shape.
test_unknown_metric_is_rejected()	—
test_a_metric_cannot_be_defined_twice()	The entire premise: one definition, one place.
test_every_catalogued_metric_compiles_and_runs(con)	—
test_every_declared_dimension_works_on_a_ratio_metric(con)	—
test_ratio_equals_numerator_over_denominator(con)	—
test_dimensional_slices_reconcile_on_the_numerator(con)	A RATIO does not sum across slices; its numerator does. Getting this
test_member_months_equal_the_sum_of_enrolment_durations(con)	The fan-out check. If a join duplicated rows, every ratio silently
test_denied_lines_never_exceed_total_lines(con)	—
test_zero_claim_member_months_stay_in_the_denominator(con)	Dropping them turns PMPM into 'average cost among members who
test_loss_ratio_is_in_a_plausible_band(con)	—
test_month_label_round_trips(m)	—
test_month_label_boundaries()	December to January is where naive month arithmetic breaks.

Opening of the module — tests/test_semantic_and_analytics.py

```
"""
test_semantic_and_analytics.py
--------------------------------
Tests for the properties that make Aegis's numbers trustworthy.

Written as invariants and boundary cases rather than as recordings of
current output, because `qa/injected_bugs.py` breaks the code on purpose
and counts what this suite notices. A test asserting
"loss_ratio == 0.8519" fails for every change and diagnoses none of them.
"""

import sys
from pathlib import Path

import duckdb
import numpy as np
import pytest
from hypothesis import given, settings
from hypothesis import strategies as st

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT / "platform" / "semantic"))
sys.path.insert(0, str(ROOT / "platform" / "ingest"))
sys.path.insert(0, str(ROOT / "analytics" / "survival"))
sys.path.insert(0, str(ROOT / "integrity"))

from metrics import REGISTRY, Metric, compile_query, metric_catalog  # noqa: E402
from domain import EM_INTENSITY, month_index, month_label  # noqa: E402
from member_survival import kaplan_meier, cox_ph, median_survival  # noqa: E402

DB = ROOT / "database" / "aegis.duckdb"
SETTINGS = settings(max_examples=100, deadline=None)


@pytest.fixture(scope="module")
def con():
    c = duckdb.connect(str(DB), read_only=True)
    yield c
    c.close()


# ================================================================
#  Semantic layer governance
# ================================================================
def test_unknown_dimension_is_rejected():
    """Governance must fail at compile time, not return a wrong shape."""
    with pytest.raises(ValueError, match="unknown dimension"):
        compile_query("loss_ratio", ["not_a_real_dimension"])


def test_unknown_metric_is_rejected():
    with pytest.raises(ValueError, match="unknown metric"):
        compile_query("invented_metric", [])


def test_a_metric_cannot_be_defined_twice():
    """The entire premise: one definition, one place."""
    with pytest.raises(ValueError, match="already defined"):
        REGISTRY.add_metric(Metric("loss_ratio", "dup", "1", None, "grain"))


def test_every_catalogued_metric_compiles_and_runs(con):
    for m in metric_catalog():
        rows = con.execute(compile_query(m["name"], [])).fetchall()
        assert len(rows) == 1, f"{m['name']} did not return a single total"


def test_every_declared_dimension_works_on_a_ratio_metric(con):
    for dim in sorted(REGISTRY.dimensions):
        rows = con.execute(compile_query("loss_ratio", [dim])).fetchall()
        assert rows, f"dimension {dim} returned nothing"
```

4.13 run_all.py

Why it exists. Every stage in order.

What the module says about itself. Builds Aegis from nothing, in dependency order, then checks it built the same thing as last time.

Public functions in run_all.py

Function	What it does
sha256(p)	—
fingerprint()	—
run(label, script)	—
main()	—

Opening of the module run_all.py

```
"""
run_all.py
----------
Builds Aegis from nothing, in dependency order, then checks it built the
same thing as last time.

    python run_all.py           full pipeline
    python run_all.py --verify   regenerate data in a NEW process, compare checksums
    python run_all.py --skip-llm skip the Ollama stage
    python run_all.py --skip-qa  skip injected-bug scoring (the slow stage)

--verify runs in a fresh process deliberately. Two earlier projects in this
portfolio shipped generators with fixed seeds that were still not
reproducible across runs:

    rng.choice(list(SOME_SET))    Project 10 - set iteration order
    abs(hash(customer_id))        Project 13 - builtin hash randomisation

Both are perfectly stable WITHIN one process, so a same-process re-run
cannot detect either. A seed is a claim about reproducibility; a checksum
taken in a new process is a measurement of it.
"""

import argparse
import hashlib
import json
import subprocess
import sys
import time
from pathlib import Path

ROOT = Path(__file__).resolve().parent
PY = sys.executable
REPORTS = ROOT / "reports"

STAGES = [
    ("Generate payer data",        "platform/ingest/generate_payer_data.py",     "generate"),
    ("Build warehouse",            "platform/warehouse/build_warehouse.py",      "warehouse"),
    ("Metric regression testing",  "integrity/metric_regression.py",             "analysis"),
    ("Member survival analysis",   "analytics/survival/member_survival.py",      "analysis"),
    ("Hierarchical forecasting",   "analytics/forecasting/hierarchical_forecast.py", "analysis"),
    ("Upcoding + red team",        "integrity/upcoding_detection.py",            "analysis"),
    ("Risk adjustment",            "analytics/risk/risk_adjustment.py",          "analysis"),
    ("Metric copilot (local LLM)", "ai/metric_copilot.py",                       "llm"),
    ("Validation & injected bugs", "qa/validation.py",                           "qa"),
]

CHECKSUM_GLOBS = ["data/**/*.csv", "data/**/*.json"]
DATA_STAGES = {"generate"}


def sha256(p):
    h = hashlib.sha256()
    with open(p, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()


def fingerprint():
    out = {}
    for pat in CHECKSUM_GLOBS:
        for p in sorted(ROOT.glob(pat)):
            out[str(p.relative_to(ROOT)).replace("\\", "/")] = sha256(p)
    return out


def run(label, script):
    print(f"\n{'=' * 72}\n{label}\n{'=' * 72}", flush=True)
    t0 = time.perf_counter()
    rc = subprocess.run([PY, str(ROOT / script)], cwd=ROOT).returncode
    secs = round(time.perf_counter() - t0, 1)
```

CHAPTER 5

Churn Is Not a Yes/No Question

> *Censoring, Kaplan-Meier, and three estimators graded against coefficients that were injected - where the obvious one is 26 times worse.*

5.1 What a churn label throws away

Project 2 modelled churn as a binary label: did this customer leave within the window? That framing is standard, and it discards two things.

1. **When.** A member who lapsed in month 2 and one who lapsed in month 22 get the same label, though they are entirely different commercial outcomes.
2. **Censoring.** A member who has been enrolled 4 months and has not lapsed is labelled 'did not churn' - but they have only been observed for 4 months. They are not a negative example; they are an *incomplete* one.

6.1 The cohort

Quantity	Value
Members	24,000
Events (lapses observed)	15,446
Censoring rate	**35.64%**
Median survival	11 months

A THIRD OF THE COHORT IS CENSORED, AND A BINARY LABEL LIES ABOUT ALL OF THEM

8,554 members had not lapsed when observation ended. Labelling them 'retained' asserts something the data does not contain - most of them will lapse eventually, and the model is being told they did not.

Survival analysis exists because 'has not happened yet' and 'did not happen' are different facts, and treating them as the same biases every coefficient toward zero in a way no amount of data fixes.

5.2 Kaplan-Meier

The survival function estimated non-parametrically, using each member for exactly as long as they were observed:

$$\hat{S}(t) = \prod_{t_i \le t} \left(1 - \frac{d_i}{n_i}\right)$$

where d_i is the number of events at time t_i and n_i the number still at risk. A censored member contributes to n_i until they leave observation and then simply drops out - no assumption is made about what happened to them afterwards.

6.2 The survival curve at selected months

Month	Survival	At risk
0	0.9184	24,000
6	0.6245	15,873
11	**0.4900**	12,306
18	0.3719	7,022
23	0.3176	1,041

WHY THE INTERVAL WIDENS AS THE CURVE FALLS

Greenwood's formula gives the variance, and it depends on the number still at risk:

$$\widehat{\mathrm{Var}}(\hat{S}(t)) = \hat{S}(t)^2 \sum_{t_i \le t} \frac{d_i}{n_i(n_i - d_i)}$$

At month 23 only 1,041 members remain at risk against 24,000 at the start, so each additional event moves the estimate much further. **The right-hand tail of a survival curve is always the least reliable part**, and quoting a median from it - or worse, extrapolating past it - is where survival analysis is most often misused.

The log-rank test comparing individual against employer channel gives $\chi^2 = 187.08$, $p \approx 0$ - the two channels have genuinely different retention curves, not a difference in their medians alone.

5.3 Three estimators against injected truth

The generator injects known coefficients, so each estimator can be graded rather than merely compared.

6.3 Coefficient recovery, graded against the injected truth

Covariate	True	Naive logistic	Cox PH	Discrete-time hazard
premium_burden_x10	0.95	1.5698	0.7720	**0.9518**
had_denial	0.55	0.4767	0.2123	**0.5329**
utilisation_low	0.40	0.6720	0.3683	**0.4170**
channel_individual	0.35	0.5709	0.3180	**0.3595**
tenure_at_month	-0.03	**cannot**	**cannot**	**-0.0188**
Mean absolute error		**0.2965**	**0.1449**	**0.0113**

THE DISCRETE-TIME HAZARD IS 26X MORE ACCURATE THAN THE NAIVE LABEL AND 13X MORE ACCURATE THAN COX

Mean absolute error 0.0113 against 0.2965 for naive logistic and 0.1449 for Cox. On `premium_burden` the naive model estimates 1.57 against a true 0.95 - **65% too large** - because censored members are being counted as retained, so the covariates that predict early lapse are over-weighted.

Cox is far better and systematically *under*-estimates: `had_denial` comes out at 0.21 against a true 0.55.

The column neither of the others can fill

`tenure_at_month` is **time-varying** - a member's tenure changes every month they remain enrolled. A binary logistic model has no place to put it. Standard Cox assumes proportional hazards with time-fixed covariates and cannot estimate it either.

PERSON-MONTH EXPANSION IS WHAT MAKES IT ESTIMABLE

The discrete-time hazard model reshapes the data to one row per member per month at risk, then fits an ordinary logistic regression to the probability of lapsing *in that month, given survival to it*.

$$h(t \mid x) = P(T = t \mid T \geq t,\, x) = \sigma(\alpha_t + \beta^\top x_t)$$

Because each row is a month, any covariate can vary by row - tenure, a denial that happened last month, a premium change. **The reshaping costs nothing but rows and buys the entire class of time-varying covariates**, which is why it recovers a coefficient the other two cannot express.

The survival module — analytics/survival/member_survival.py

```
"""
member_survival.py
--------------------
Time-to-Lapse analysis on enrolment spans where 35.6% of members have NOT
lapsed by the time we look.

Those members are the whole point. They are RIGHT-CENSORED: their lapse
time is unknown and greater than their observed tenure. The instinctive
move - label them "did not churn" and fit a classifier - throws away the
distinction between "stayed 22 months and is still here" and "stayed 2
months and is still here", and treats the second as evidence of loyalty.

Four estimators are fitted on identical data and scored against the hazard
the generator injected:

  NAIVE LOGISTIC     churned yes/no, ignoring tenure entirely. The control
                     arm, and the thing most teams actually ship.
  KAPLAN-MEIER       non-parametric survival by cohort, with Greenwood
                     confidence bands and at-risk counts.
  COX PH             partial likelihood (Breslow ties), hand-implemented.
                     The right family; a continuous-time approximation to
                     a process that is actually discrete.
  DISCRETE-TIME      person-month logistic hazard. This one MATCHES the
  HAZARD             data-generating process exactly, and is included
                     because "Cox is the survival model" is a habit rather
                     than a decision - when data arrive in monthly
                     buckets, the discrete-time model is the correct one.

Everything is implemented directly rather than through lifelines: the
partial likelihood is fifteen lines, and writing it is the difference
between using survival analysis and understanding it. It also keeps the
project runnable with no extra dependency.

Output: reports/survival.json
"""

import json
import sys
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.optimize import minimize
from sklearn.linear_model import LogisticRegression

ROOT = Path(__file__).resolve().parent.parent.parent
sys.path.insert(0, str(ROOT / "platform" / "ingest"))
from domain import LAPSE_COEFFS, N_MONTHS  # noqa: E402

DATA = ROOT / "data"
REPORTS = ROOT / "reports"
REPORTS.mkdir(exist_ok=True)

# The covariates the generator actually used. Named here so the scoring is
# a like-for-like comparison rather than a fishing expedition.
COVARIATES = ["premium_burden_x10", "had_denial", "utilisation_low",
              "channel_individual"]


def load():
```

CHAPTER 6

Competing Risks

> *Why one minus the survival curve overstates the thing you care about by ten percentage points.*

6.1 Not every exit is the same exit

7.1 How members actually leave

Exit cause	Share
Voluntary lapse	**62.1%**
Switched plan	26.8%
Aged out	7.9%
Deceased	3.2%

A retention team cares about **voluntary lapse** - the exits they could have prevented. The other three are not retention failures.

THE OBVIOUS CALCULATION IS WRONG, AND WRONG IN A PREDICTABLE DIRECTION

Fit a Kaplan-Meier curve to voluntary lapse, treating the other exits as censoring, then report $1 - \hat{S}(t)$ as the cumulative lapse probability. This is standard practice and it is biased upward.

Censoring assumes the member *could still have experienced the event later*. A deceased member cannot voluntarily lapse. Neither can one who already switched plans. **Treating a competing event as censoring asserts a counterfactual that is impossible.**

6.2 The measured overstatement

7.2 Two ways to compute the same quantity

Estimate	Value
`1 - KM` for voluntary lapse	**0.5053**
Cumulative incidence function	**0.3996**
Overstatement	**0.1057**

$$\mathrm{CIF}_k(t) = \sum_{t_i \le t} \hat{S}(t_{i-1}) \cdot \frac{d_{k,i}}{n_i}$$

The cumulative incidence function weights each cause-specific hazard by the probability of still being *event-free of everything* at that point. It cannot exceed the overall event probability, and the cause-specific CIFs sum to it.

TEN AND A HALF PERCENTAGE POINTS, A 26% RELATIVE OVERSTATEMENT

0.5053 against 0.3996. The project's own note states the mechanism: *"1-KM treats death and plan-switch as censoring, which assumes those members could still have lapsed later. They could not. The cumulative incidence function is the honest number and is always smaller."*

The direction is guaranteed, not incidental - 1-KM is always an overstatement in the presence of competing risks, and the size of the error grows with the share of exits that are competing events. Here 37.9% of exits are not voluntary lapses, which is why the gap is large.

A retention team acting on 50.5% would size their intervention budget against a population a quarter larger than the one that exists - and would then report an improvement when the true number came in lower.

CHAPTER 7

Detection Against an Adversary Who Adapts

> *Three rules, one of which never fires, and what happens to each when the target changes behaviour.*

7.1 The detector

10.1 The three signals

Signal	Rule	Rationale
`peer_intensity`	z > 2.5 within specialty	a cardiologist bills differently from a GP; compare like with like
`top_code_share`	z > 2.5 on share of top-two codes	upcoding concentrates billing on the highest-paying codes
`impossible_day`	> 600 billed minutes in a day	a physical impossibility, not a statistical anomaly

A volume floor excludes providers with fewer than 40 claims. The project's stated reason: *"a provider with a handful of claims has a mean intensity that is noise; flagging them generates false positives an investigator cannot act on."*

WITHIN-SPECIALTY COMPARISON IS WHAT MAKES THE Z-SCORE MEANINGFUL

Billing intensity varies enormously across specialties for entirely legitimate reasons. A global z-score would flag every high-intensity specialty and miss the outlier inside a low-intensity one.

This is the same instinct as Project 6 scoring each account against its own history rather than a population threshold - **the comparison group is the design decision that determines what the detector can see.**

7.2 The baseline result

10.2 Against an unaware adversary

Metric	Value
Professional claim lines	124,508
Providers scored	900
Fraudulent providers in scope	22
Flagged	23
True positives	21
False positives	2
Precision / Recall / F1	**0.913 / 0.9545 / 0.9333**

The rule that never fires

10.3 Rule hit counts

Rule	Hits at baseline	Hits under every red-team scenario
`peer_intensity`	22	5 to 22
`top_code_share`	20	7 to 20
`impossible_day`	**0**	**0**

ONE OF THE THREE SIGNALS CONTRIBUTED NOTHING, IN ANY SCENARIO

`impossible_day` - more than 600 billed minutes in a day - never fired once. It is the most intuitively compelling rule of the three and it caught nobody.

That is not a bug; it is what an unfired rule looks like. The fraud in this data is subtle upcoding rather than crude volume fabrication, so a rule targeting the crude version has nothing to find. Worth keeping - it costs nothing and covers a real attack - and worth knowing that it is contributing zero to the headline number.

7.3 Three adaptations

The red team asks the only question that matters for a control: what does it cost the adversary to evade it?

10.4 The full red-team result

Scenario	Change	Recall	Precision	Recall drop
baseline	-	0.9545	0.913	-
stay_below_threshold	never bill the top code	0.7727	-	0.1818
spread_thin	half the ring stops	0.5000	-	0.4545
reduced_rate	**upcode 15% not 42%**	**0.1818**	0.5714	**0.7727**

WHY REDUCING THE RATE IS SO MUCH MORE EFFECTIVE THAN THE OTHERS

Both surviving rules are **z-scores within specialty**. A z-score measures distance from the peer mean in units of peer standard deviation:

$$z = \frac{\bar{x}_{\text{provider}} - \mu_{\text{specialty}}}{\sigma_{\text{specialty}}}$$

Upcoding 42% of claims moves the provider's mean intensity far enough to clear $z > 2.5$. Upcoding 15% moves it roughly a third as far - **and the threshold is a cliff, not a slope.** A provider at z = 2.4 is invisible; one at z = 2.6 is flagged, and the fraud is nearly as profitable at the former.

Note also what happened to precision: it falls from 0.913 to 0.5714 in the reduced-rate scenario. The detector does not merely miss more - **the flags it does raise become less trustworthy**, because the genuine fraudsters have moved into the same z-range as the legitimate outliers.

7.4 What a control is worth against an adaptive opponent

1. **Report the adversarial number, not the baseline.** 95.45% describes a world with no fraudsters who have noticed the control. 18.18% describes the world after one of them thinks for an afternoon.
2. **Thresholds are public in effect.** A fraudster can discover `z > 2.5` by probing - submit, observe whether an audit follows, adjust. Once known, it defines a safe operating region.
3. **Cliff-edge rules invite exactly this.** A hard threshold means the adversary's optimal strategy is to sit just below it. A graduated response - higher scrutiny with higher z, rather than a binary flag - removes the cliff.
4. **The economics decide.** The question is never 'can this be evaded' but 'what does evasion cost relative to the fraud's value'. Here it costs two-thirds of the fraudulent volume for a five-fold reduction in detection - a trade any rational adversary takes.

WHICH IS THE SAME FINDING AS PROJECT 6, MEASURED PROPERLY

Project 6's velocity rule was blind to the first three events of any burst, so an attacker sending bursts of three was *completely invisible* - a structural result derived from the rule.

Here the result is empirical rather than structural, and the shape is identical: **any detector with a fixed threshold on a quantity the adversary controls has a region in which the adversary is safe by construction.** The defence is not a better threshold but a signal the adversary cannot cheaply manipulate.

The detector and its red team — integrity/upcoding_detection.py

```
"""
upcoding_detection.py
-----------------------
Finding providers who bill a higher-intensity E/M code than the encounter
justifies, and then finding out how much of that detection survives an
adversary who knows the rules.

Detection uses BEHAVIOUR only. `is_fraud` exists in the provider table and
is used solely to score - never as a feature, never as a filter. The same
discipline applies to `true_em_code`, which lives in a separate scoring
file so it cannot be joined in by accident.

Three signals, each with an independent rationale:

  PEER INTENSITY   a provider's mean E/M level against peers in the SAME
                   specialty. Specialty matters: an oncologist legitimately
                   bills higher than a family doctor, and comparing across
                   specialties would flag the entire oncology department.
  DISTRIBUTION     the share of a provider's claims at the top two levels.
                   Catches a provider whose mean looks normal because they
                   balance high codes with low ones.
  IMPOSSIBLE DAY   billed encounter minutes exceeding a plausible working
                   day. A hard rule with no statistics in it, included
                   because it is the one an investigator can defend without
                   explaining a z-score to a lawyer.

THE RED TEAM
------------
A detector's recall against a static adversary is close to meaningless,
because the adversary is not static. Once thresholds are known, upcoding
adapts: bill just under the flagging line, spread the behaviour across
more encounters, or shift only within the codes where peers are diffuse.
Each evasion is simulated and recall is re-measured, so the reported
strength is strength under attack rather than strength on the training
distribution.

Output: reports/upcoding_detection.json
"""

import json
import sys
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT / "platform" / "ingest"))
from domain import (  # noqa: E402
    EM_CODES, EM_INTENSITY, EM_MINUTES, IMPOSSIBLE_DAY_MINUTES, UPCODE_SHIFT,
)

DATA = ROOT / "data"
REPORTS = ROOT / "reports"
REPORTS.mkdir(exist_ok=True)

MIN_CLAIMS_FOR_SCORING = 40      # below this a provider's mean is noise
FLAG_Z = 2.5                     # peer-relative z-score threshold
TOP_CODE_SHARE_Z = 2.5


def load():
    claims = pd.read_csv(DATA / "claim_lines.csv", dtype={"em_code": "string"})
    providers = pd.read_csv(DATA / "providers.csv")
    prof = claims[claims.service_category == "professional"].copy()
    prof = prof[prof.em_code.notna()]
    prof["intensity"] = prof.em_code.map(EM_INTENSITY)
    prof["minutes"] = prof.em_code.map(EM_MINUTES)
    return prof, providers
```

And the generator that plants the ring the detector is graded against, including the parameters each adversarial scenario varies:

The seeded payer generator — platform/ingest/generate_payer_data.py

```
"""
generate_payer_data.py
-------------------------
Generates the Aegis book of business, with every injected mechanism
recorded so downstream analysis can be scored rather than admired.

The design point that matters most is CENSORING. Members are simulated
month by month with a real hazard of lapsing, and the observation window
closes at OBSERVATION_END_MONTH. A member still enrolled at that point has
not "not churned" - their lapse time is simply unknown and greater than
their observed tenure. Recording `event_observed = 0` for them is what
makes survival analysis possible and what makes the naive
churned-yes/no control arm measurably wrong.

Outputs:
    data/members.csv              one row per member (demographics, plan)
    data/enrollment_spans.csv     tenure + event/censoring flag
    data/providers.csv
    data/claim_lines.csv          the fact table
    data/clinical_notes.csv       free text for the AI layer
    data/ground_truth_payer.json  the answer key
"""

import json
import sys
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parent.parent.parent
sys.path.insert(0, str(Path(__file__).resolve().parent))
from domain import (  # noqa: E402
    BASELINE_MONTHLY_HAZARD, CHANNELS, CLAIM_STATUS, DENIAL_REASONS,
    EM_ALLOWED_AMOUNT, EM_CODES, EM_INTENSITY, EM_MINUTES,
    FRAUD_RING_SPECIALTIES, LAPSE_CAUSE_SHARES, LAPSE_COEFFS,
    LEVEL_SHIFT_MONTH, LEVEL_SHIFT_REGION, LEVEL_SHIFT_SIZE, N_MEMBERS,
    N_MONTHS, N_FRAUD_PROVIDERS, N_PROVIDERS, NULL_PROXY_NAME,
    NULL_PROXY_TRUE_EFFECT, OBSERVATION_END_MONTH, PLACE_OF_SERVICE, PLANS,
    PRODUCTS, REGION_MONTHLY_TREND, REGIONS, RISK_COEFFS, SEASONAL_AMPLITUDE,
    SEED_CLAIMS, SEED_FRAUD, SEED_LAPSE, SEED_MEMBERS, SEED_NOTES,
    SEED_PROVIDERS, SERVICE_CATEGORIES, SPECIALTIES, UPCODE_SHIFT, COST_SCALE,
    month_label,
)

DATA = ROOT / "data"
DATA.mkdir(exist_ok=True)

PLAN_PREMIUM = {"Bronze": 310.0, "Silver": 430.0, "Gold": 585.0, "Platinum": 760.0}
PLAN_ACTUARIAL_VALUE = {"Bronze": 0.60, "Silver": 0.70, "Gold": 0.80, "Platinum": 0.90}


# ======================================================================
#  Members
# ======================================================================
def generate_members(rng):
    n = N_MEMBERS
    plan = rng.choice(PLANS, size=n, p=[0.28, 0.37, 0.24, 0.11])
    product = rng.choice(PRODUCTS, size=n, p=[0.44, 0.41, 0.15])
    region = rng.choice(REGIONS, size=n, p=[0.24, 0.21, 0.19, 0.20, 0.16])
    channel = rng.choice(CHANNELS, size=n, p=[0.52, 0.21, 0.19, 0.08])

    age = np.clip(rng.normal(44, 16, n), 18, 89).astype(int)
    income = np.clip(rng.lognormal(10.9, 0.52, n), 14_000, 400_000)
    chronic = rng.poisson(np.clip(0.35 + (age - 40) * 0.021, 0.05, None), n)
```

CHAPTER 8

Risk Adjustment and Calibration

> *Predicting next year's cost, why the time split is not optional, and why a linear model beat gradient boosting again.*

8.1 The split decides the result

Features come from months 0-11; the target is cost in months 12 onward. The project states why in its own report:

The recorded reason for the time split

```
"a random split over member-months lets a member's later cost inform
 their earlier features; cost is highly autocorrelated so every metric
 inflates"
```

THIS IS PROJECT 13'S 2X2 FINDING, APPLIED BEFORE THE FACT

Project 13 measured what a random split does to a rare-event model: PR-AUC 0.8495 under a random split against 0.0352 under a temporal one, with identical features. A 24-fold difference caused entirely by the split.

Here the lesson is applied rather than rediscovered. The split is temporal by design and the reason is written into the artefact, which is what a portfolio is supposed to accumulate.

8.2 Three models

8.1 Risk adjustment models, time-split

Model	Top-decile lift	R^2	MAE	Calibration slope	Calibrated?
mean_baseline	1.066	-0.000	2,862	-	no
ridge	**1.834**	**0.1254**	**2,603**	1.0418	**yes**
gradient_boosting	1.827	0.1127	2,619	0.9670	**yes**

RIDGE REGRESSION BEAT GRADIENT BOOSTING - THE FIFTH TIME IN THIS PORTFOLIO

R^2 0.1254 against 0.1127, MAE 2,603 against 2,619, and a marginally better top-decile lift. A penalised linear model outperformed a gradient-boosted ensemble on tabular data with engineered features.

Project 3's seasonal naive beat SARIMA. Project 7's popularity baseline beat matrix factorisation. Project 11's logistic regression matched gradient boosting. Project 12's bigram matched a 22.7-million-parameter transformer. **Five projects, five domains, one pattern - and each was only visible because the baseline was built first.**

And R^2 of 0.125 is the honest headline

Both models explain about an eighth of the variance in next-year cost. That is not a failure - individual healthcare spending is dominated by unpredictable events, and a top-decile lift of 1.83 is genuinely useful for targeting care management. But a reader should not leave with the impression that cost is predictable.

8.3 Calibration matters more than discrimination here

Risk adjustment does not rank members for a decision - it sets *payments*. A model that ranks perfectly and predicts systematically low transfers the wrong amount of money.

$$\text{actual} = \alpha + \beta \cdot \text{predicted}$$

A calibration slope $\beta = 1$ means predictions are correct on average at every level. Ridge gives 1.0418; gradient boosting 0.9670.

8.2 Calibration, and what it costs

Model	Slope	Reading	Aggregate payment error
ridge	1.0418	slightly under-predicts the high end	-0.735%
gradient_boosting	0.9670	slightly over-predicts the high end	-0.587%

THE TWO ERRORS ARE NOT SYMMETRIC IN A PAYMENT SYSTEM

A slope above 1 means the model under-predicts for high-cost members - so plans enrolling the sickest members are systematically underpaid, which creates a financial incentive to avoid them. A slope below 1 over-pays for high risk, which invites the upcoding that Chapter 10 detects.

The calibration slope is not a diagnostic here; it is a policy parameter with behavioural consequences - and an aggregate payment error under 1% is the number a regulator would actually check.

Note that the mean baseline has no calibration slope at all - it predicts one value for everyone, so there is no line to fit. Its top-decile lift of 1.066 is the floor any model must beat, and it is not 1.000 only because the deciles are formed on a constant and therefore arbitrary.

CHAPTER 9

Hierarchical Forecasting, and Reconciliation That Lost

> *Forecasts that do not add up, four ways to make them coherent, and the two sophisticated methods that came last.*

9.1 Forecast PMPM, not totals

The project's stated reason is the most important design decision in the chapter:

Why the target is per-member-per-month

```
"total cost confounds price with membership; this book shrinks as
 members lapse, so a forecaster trained on totals would project the
 shrinkage as a cost improvement"
```

A FORECASTER ON TOTALS WOULD REPORT A SHRINKING BUSINESS AS A COST WIN

Chapter 6 established that this membership base is lapsing - median survival is 11 months. Total paid cost therefore falls, and a model forecasting totals would project the fall and present it as improving cost performance.

The metric would be correct and the interpretation catastrophic. Normalising by member months separates price from volume, which is what makes the forecast about cost rather than about attrition.

9.2 Coherence

Twenty-four series across four levels - total, region, product, leaf - with 15 leaves. Forecast each independently and they will not add up.

9.1 What independent forecasting costs

Quantity	Value
Base forecast incoherence	**690.60**
As a share of the forecast	0.486%
Meaning	the gap between the regional numbers and the total in the same board pack

INCOHERENCE IS A CREDIBILITY PROBLEM BEFORE IT IS A STATISTICAL ONE

A board pack where the regions sum to 690 more than the stated total invites exactly one question, and no answer to it is good. Reconciliation exists as much to make the numbers *defensible* as to make them accurate.

9.3 Four reconciliation methods

9.2 Every approach, on a 6-month holdout

Approach	MAPE	RMSE	Incoherence
base_independent	13.479%	132.27	**690.60**
bottom_up	13.341%	128.63	0
top_down	**9.545%**	**99.44**	0
ols_reconciled	**13.568%**	131.89	0
mint_diagonal	**13.586%**	131.47	0

THE TWO SOPHISTICATED METHODS ARE WORSE THAN DOING NOTHING

OLS reconciliation (13.568%) and MinT (13.586%) both score *worse* than the incoherent base forecast (13.479%). They buy coherence and pay for it in accuracy.

Top-down wins outright at **9.545%** - a 29% relative improvement over base - and is coherent for free. **The simplest reconciliation beat the two methods designed to be optimal.**

Why top-down wins here

Look at the per-level errors of the base forecast: total 4.27%, region 7.46%, product 10.59%, leaf 16.68%. **The aggregate is far easier to forecast than the leaves**, because idiosyncratic noise cancels when series are summed.

Top-down forecasts only the total - the one series with 4.27% error - and distributes it by historical proportions. It never fits the noisy leaf series at all, so it never inherits their 16.68%.

> **AND WHY MINT SHOULD HAVE WON, IN THEORY**
>
> MinT reconciles by minimising the trace of the forecast error covariance, and with correctly estimated covariances it is provably optimal. The diagonal variant used here assumes the errors are uncorrelated across series.
>
> With 17 training months and 24 series, that covariance is estimated from very little data, and a badly estimated covariance produces weights that are worse than not weighting at all. **An optimal method with a badly estimated nuisance parameter loses to a crude one** - the same reason Project 12's IPS lost to a direct method at an effective sample fraction of 0.176.

The hierarchical forecaster — analytics/forecasting/hierarchical_forecast.py

```
"""
hierarchical_forecast.py
--------------------------
Forecasting PMPM across a hierarchy whose levels must sum.

The planning hierarchy is region x product, aggregating to region, to
product, and to the total. Forecasts are produced independently at each
level because different people own them - and independent forecasts do
not add up. The regional numbers will not sum to the total, and both
appear in the same board pack.

That inconsistency is not cosmetic. It means at least one of the numbers
someone is being held to is not the number the plan was built on.

Reconciliation fixes it, and the interesting question is whether it costs
anything:

  BASE          independent forecasts per series. Incoherent by
                construction; the incoherence is measured.
  BOTTOM-UP     forecast the leaves, sum upward. Always coherent, and
                throws away whatever signal the aggregate series had -
                aggregates are less noisy than their parts.
  TOP-DOWN      forecast the total, split by historical share. Coherent,
                stable at the top, and unable to represent a leaf
                diverging from its siblings.
  OLS / MinT    project the base forecasts onto the coherent subspace,
                using all levels at once. Should beat both, and this file
                measures whether it actually does rather than assuming it.

WHY PMPM AND NOT TOTAL COST
--------------------------
Total claims cost confounds price with membership. This book shrinks as
members lapse, so total cost can fall while cost per member rises - and a
forecaster trained on totals would project the shrinkage forward as if it
were a cost improvement. PMPM is what actuaries forecast, and it is the
target here for that reason.

Output: reports/hierarchical_forecast.json
"""

import json
import sys
from pathlib import Path

import duckdb
import numpy as np

ROOT = Path(__file__).resolve().parent.parent.parent
sys.path.insert(0, str(ROOT / "platform" / "ingest"))
from domain import (  # noqa: E402
    LEVEL_SHIFT_MONTH, LEVEL_SHIFT_REGION, LEVEL_SHIFT_SIZE, N_MONTHS,
    REGION_MONTHLY_TREND, SEASONAL_AMPLITUDE,
)

DB_PATH = ROOT / "database" / "aegis.duckdb"
REPORTS = ROOT / "reports"
REPORTS.mkdir(exist_ok=True)

HOLDOUT = 6                # forecast the final 6 months
SEASON = 12
```

The honest recommendation: on this data, use top-down. State the condition - it works because the total is much more forecastable than the leaves - and re-check it if the hierarchy or the history length changes.

CHAPTER 10

A Copilot That Cannot Write SQL

> *Constraining a model to resolve rather than generate, and a refusal rate of zero on questions with no answer.*

10.1 The design

From the project's own report:

The copilot's architecture

```
"questions resolve to a governed metric + declared dimensions; the
 platform compiles the SQL. The model never writes SQL, so it cannot
 invent a definition, join at the wrong grain, or read a column it
 should not."
```

THIS IS THE STRONGEST VERSION OF THE PATTERN THE WHOLE PORTFOLIO USES

Project 9's NL-to-SQL let the model write queries and guarded them syntactically - which passed a PII-extraction request because it was a valid SELECT. Here the model's entire output is a metric name and a list of dimensions, both from a closed vocabulary of seven metrics and six dimensions.

The attack surface is not narrowed, it is removed. There is no string the model can emit that becomes SQL, so injection, wrong grain and undeclared columns are all structurally impossible rather than defended against.

10.2 The measurement

12.1 16 questions, of which 4 are unanswerable

	Rules baseline	LLM (qwen2.5:0.5b)
Overall accuracy	**0.875**	**0.250**
Resolution accuracy	0.9167	0.3333
Refusal accuracy	**0.750**	**0.000**

THE MODEL REFUSED NONE OF THE FOUR UNANSWERABLE QUESTIONS

Four questions cannot be answered from the governed vocabulary - they ask for a metric that does not exist or a dimension that is not declared. The correct response is to decline.

The model declined zero of them, and all 16 of its resolutions executed successfully. It confidently resolved every unanswerable question to *some* metric and dimension, and the platform dutifully compiled and ran the query.

So the user asks a question the system cannot answer and receives a number. Not an error - a plausible, well-formatted, wrong-question answer.

Which is exactly why the constrained design matters

Note the consolation: because the model can only emit a metric name and dimensions, the wrong answer is still a *correctly computed governed metric*. It answers the wrong question, and it does not invent a definition, leak a column or produce a number that is internally wrong.

THE RULES BASELINE IS NOT PERFECT EITHER

0.875 overall, and refusal accuracy of 0.750 - it also gets one of the four unanswerable questions wrong. The gap that matters is 0.750 against 0.000: **a keyword matcher that fails to match declines by default, and a language model always has something to say.**

This is the sixth measurement in this portfolio of the same 0.5B model failing a judgement task - zero of eleven PII instances in Project 9, 2 of 5 NL-to-SQL, 0.333 intent classification in Project 10, 8 of 36 alert triage in Project 14, and now 0.25 here. The envelope is consistent and it is narrow.

The copilot — ai/metric_copilot.py

```
"""
metric_copilot.py
-------------------
Natural-language questions answered through the SEMANTIC LAYER, not
through generated SQL.

The usual text-to-SQL design hands a model the schema and asks for a
query. On a payer's warehouse that is close to the worst available idea:
the model can invent a definition of loss ratio, join at the wrong grain,
or read a column it has no business reading, and the answer comes back
formatted confidently either way.

Here the model does something much smaller and much safer. It maps the
question onto a CLOSED VOCABULARY - one of seven governed metrics and a
handful of declared dimensions - and the platform compiles the query
itself. The model cannot express a wrong definition because it never
writes a definition. The worst it can do is pick the wrong metric, and
that is both detectable and recoverable.

Three things are measured:

  RESOLUTION ACCURACY   does it pick the right metric and dimensions,
                        scored against an answer key of questions?
  RULES BASELINE        keyword matching, which is what this would be
                        without a model. If the model cannot beat it, the
                        model is decoration.
  REFUSAL               questions the vocabulary genuinely cannot answer
                        must be REFUSED, not answered with the nearest
                        available metric. An analytics assistant that
                        always produces a number is more dangerous than
                        one that sometimes says no.

Runs on local Ollama qwen2.5:0.5b - free, offline, no expiry.

Output: reports/metric_copilot.json
"""

import json
import re
import sys
import time
import urllib.error
import urllib.request
from pathlib import Path

import duckdb

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT / "platform" / "semantic"))
from metrics import REGISTRY, compile_query, metric_catalog  # noqa: E402

DB_PATH = ROOT / "database" / "aegis.duckdb"
REPORTS = ROOT / "reports"
REPORTS.mkdir(exist_ok=True)
```

CHAPTER 11

Worked Examples

> *Kaplan-Meier by hand, the CIF correction, the coefficient recovery, and the adversary's arithmetic.*

11.1 Example 1 - Kaplan-Meier by hand

Ten members. Events at months 2, 3 and 5; censoring at month 4.

13.1 The product-limit estimator, step by step

<img class="eq-inline" src=" alt="t">
2
3
4
5

THE CENSORED MEMBER REDUCES THE RISK SET AND NEVER CONTRIBUTES AN EVENT

At month 4 one member is censored. $\hat{S}$ does not move - there was no event - but the risk set falls from 7 to 6, so the single event at month 5 now carries more weight.

That is the entire mechanism. The member contributed four months of information and no assumption was made about month five onward. A binary label would have called them 'retained' and thrown the distinction away.

11.2 Example 2 - Why 1-KM overstates

Suppose 100 members, and in the first period 10 lapse and 10 die.

13.2 The same data, two assumptions

Method	Treats deaths as	Lapse probability
1 - KM	censored - could still lapse later	**higher**
CIF	a competing event - cannot lapse	**lower**

1-KM computes the lapse hazard among those at risk and then extrapolates as if the deceased were still eligible to lapse in later periods. They are not, so the survival-weighted sum is inflated.

$$1 - \hat{S}_{\text{lapse}} = 0.5053 \quad \text{vs} \quad \text{CIF}_{\text{lapse}} = 0.3996$$

$$\text{overstatement} = 0.1057 \;\; (26.4\% \text{ relative})$$

37.9% OF EXITS ARE COMPETING EVENTS, AND THAT DRIVES THE SIZE OF THE ERROR

Voluntary lapse is 62.1% of exits; switching, ageing out and death are the other 37.9%. The larger that share, the larger the overstatement - so the bias is worst precisely in books with high non-voluntary turnover, such as senior products.

11.3 Example 3 - Where the naive coefficient goes wrong

True `premium_burden_x10` coefficient: 0.95. Naive logistic estimate: 1.5698 - **65% too large**.

13.3 One coefficient, three estimators

Estimator	Estimate	Error	Direction
Naive logistic	1.5698	+0.6198	over
Cox PH	0.7720	-0.1780	under
Discrete-time hazard	**0.9518**	**+0.0018**	**essentially exact**

THE NAIVE MODEL OVER-WEIGHTS EARLY LAPSERS BECAUSE IT MISLABELS LATE ONES

35.6% of members are censored and labelled 'retained'. Many of them have high premium burden and would have lapsed had observation continued. The model therefore sees high-burden members in both classes, and compensates by making the coefficient larger to separate the ones who did lapse - which happened to be the earliest, most extreme cases.

Mean absolute error across all coefficients: **0.2965 naive, 0.1449 Cox, 0.0113 discrete-time**. That is **26x** and **13x** respectively - and the discrete-time model also estimates a time-varying coefficient neither of the others can express.

11.4 Example 4 - The adversary's arithmetic

A fraudulent provider upcodes 42% of claims and is caught. What does it cost to become invisible?

13.4 Three adaptations, measured

Scenario	Behaviour change	Detector recall	Recall drop
baseline	upcode 42% of claims	**0.9545**	-
stay_below_threshold	never bill the top code	0.7727	0.1818
spread_thin	half the ring stops	0.5000	0.4545
reduced_rate	**upcode 15% instead of 42%**	**0.1818**	**0.7727**

THE CHEAPEST ADAPTATION IS THE MOST EFFECTIVE ONE

Abandoning the top code costs the fraudster their highest-margin claims and only drops recall to 0.77. Shutting down half the ring costs half the revenue and drops it to 0.50.

Simply upcoding less - 15% of claims instead of 42% - drops detection to 0.1818. The fraudster keeps the same scheme, the same codes and the same specialty, gives up roughly two-thirds of the fraudulent volume, and becomes 5 times harder to catch.

The economics decide the outcome. A provider retaining 36% of their fraudulent revenue at a 5-fold reduction in detection probability has made an excellent trade, and nothing about the detector's baseline performance predicts it.

CHAPTER 12

Running the Project, Step by Step

> *Every terminal command needed to go from a fresh clone to a finished run, what each one does, and what to do when one of them fails.*

12.1 Prerequisites

You need Python 3.11 or newer and Git. Verify both before starting - most reported 'the project does not work' turns out to be an environment that was never activated.

Check your toolchain

```
python --version
git --version
```

THIS PROJECT HAS ONE OPTIONAL LOCAL-LLM STAGE

Install Ollama and pull the model if you want the generative feature to run. It is optional: the project completes without it, and the deterministic control arm still produces results.

Optional - only for the LLM stage

```
ollama pull qwen2.5:0.5b
ollama list
```

12.2 Step 1 - Clone and enter the repository

Get the code

```
git clone https://github.com/Nikhil201716/15-Aegis-Health-Plan-Platform.git
cd 15-Aegis-Health-Plan-Platform
```

12.3 Step 2 - Create an isolated environment

A virtual environment keeps this project's dependencies from colliding with anything else on your machine.

Create and activate

```
python -m venv .venv

# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate
```

IF POWERSHELL REFUSES TO RUN THE ACTIVATION SCRIPT

Windows blocks unsigned scripts by default. Allow them for your own user account only:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

12.4 Step 3 - Install dependencies

Install

```
pip install -r requirements.txt
```

12.5 Step 4 - Run the pipeline, stage by stage

Each command below produces the input the next one consumes. Run them in this order:

The full pipeline in order

```
python platform/ingest/generate_payer_data.py
python platform/warehouse/build_warehouse.py
python integrity/metric_regression.py
python integrity/upcoding_detection.py
python analytics/survival/member_survival.py
python analytics/risk/risk_adjustment.py
python analytics/forecasting/hierarchical_forecast.py
python ai/metric_copilot.py
python qa/validation.py
```

What each stage does:

1. `platform/ingest/generate_payer_data.py` - writes the seeded payer book - members, enrolment spells, claims and providers - with the injected survival coefficients, upcoding ring and competing exit causes
2. `platform/warehouse/build_warehouse.py` - builds the warehouse the semantic layer compiles against
3. `integrity/metric_regression.py` - snapshots the 50 golden values, checks for drift, and computes the blast radius of a definition change - this is the stage that produces the 33-of-50 result
4. `integrity/upcoding_detection.py` - scores 900 providers against the three signals, then re-runs the detector against three adversarial adaptations
5. `analytics/survival/member_survival.py` - Kaplan-Meier, the log-rank test, three estimators graded against the injected coefficients, and the competing-risks correction
6. `analytics/risk/risk_adjustment.py` - time-split cost prediction with calibration measured by decile
7. `analytics/forecasting/hierarchical_forecast.py` - four reconciliation approaches over the 24-series hierarchy
8. `ai/metric_copilot.py` - resolves 16 questions - four of them unanswerable - against the governed vocabulary
9. `qa/validation.py` - builds the requirements traceability matrix and reports its gaps

EXPECTED OUTPUT

Several results here are supposed to look bad and are the point. Upcoding recall should collapse from about 0.95 to about 0.18 under the reduced_rate scenario - if it does not, check that the red team is still regenerating claims at the lower upcoding rate rather than reusing the baseline data. The metric regression should report 33 of 50 figures moved. The copilot's LLM path should score around 0.25 with a refusal accuracy of 0.000, against a rules baseline of 0.875. The traceability matrix should report 11 of 12 with REQ-012 named as the gap. The copilot stage needs Ollama; the warehouse, integrity and analytics stages do not.

12.6 Run everything in one command

Executes every stage above in order.

```
python run_all.py
```

12.7 Reproduce the adversarial collapse

Should print 0.9545 at baseline and 0.1818 for reduced_rate. The 77.3-point drop from the cheapest available adaptation is the finding.

```
python -c "import json; d=json.load(open('reports/upcoding_detection.json')); print('baseline', d['baseline']['recall']); [print(s['scenario'], s['recall'], s['description']) for s in d['red_team']]"
```

12.8 Reproduce the metric blast radius

One definition change, 33 of 50 published figures restated - each of them named in the report so it can be checked or reissued.

```
python -c "import json; b=json.load(open('reports/metric_regression.json'))['definition_change_blast_radius']; print(b['change']); print('moved', b['figures_moved'], 'of', b['figures_compared'])"
```

12.9 Compare the three survival estimators

0.2965 naive, 0.1449 Cox, 0.0113 discrete-time hazard - 26x and 13x respectively, against coefficients the generator injected.

```
python -c "import json; e=json.load(open('reports/survival.json'))['estimators']; [print(k, 'mean abs err', v['mean_abs_error']) for k,v in e.items()]"
```

12.10 See the traceability gap

91.7%, with REQ-012 uncovered - the blast-radius publication itself, which is the capability that produced the 33-of-50 result.

```
python -c "import json; t=json.load(open('reports/validation.json'))['traceability']; print(t['coverage_pct'], '%'); print(t['gaps'])"
```

12.11 Ask the copilot something it cannot answer

There is no such governed metric. The correct response is a refusal; the LLM path refused none of the four unanswerable questions in the evaluation.

```
python ai/metric_copilot.py "What is our average member satisfaction score?"
```

12.12 Launch the payer console

Opens the console over the governed warehouse.

```
python app/api/main.py
```

12.13 Command reference card

Every command from this chapter in one block, for when you already know what you want.

Complete command reference

```
# --- setup (once) ---
git clone https://github.com/Nikhil201716/15-Aegis-Health-Plan-Platform.git
cd 15-Aegis-Health-Plan-Platform
python -m venv .venv
.venv\Scripts\Activate.ps1          # Windows
source .venv/bin/activate            # macOS / Linux
pip install -r requirements.txt

# --- run ---
python platform/ingest/generate_payer_data.py
python platform/warehouse/build_warehouse.py
python integrity/metric_regression.py
python integrity/upcoding_detection.py
python analytics/survival/member_survival.py
python analytics/risk/risk_adjustment.py
python analytics/forecasting/hierarchical_forecast.py
python ai/metric_copilot.py
python qa/validation.py

# --- extras ---
python run_all.py
python -c "import json; d=json.load(open('reports/upcoding_detection.json')); print('baseline', d['baseline']['recall']); [print(s['scenario'], s['recall'],
s['description']) for s in d['red_team']]"
python -c "import json; b=json.load(open('reports/metric_regression.json'))['definition_change_blast_radius']; print(b['change']); print('moved',
b['figures_moved'], 'of', b['figures_compared'])"
python -c "import json; e=json.load(open('reports/survival.json'))['estimators']; [print(k, 'mean abs err', v['mean_abs_error']) for k,v in e.items()]"
python -c "import json; t=json.load(open('reports/validation.json'))['traceability']; print(t['coverage_pct'], '%'); print(t['gaps'])"
python ai/metric_copilot.py "What is our average member satisfaction score?"
python app/api/main.py

# --- finish ---
deactivate
```

12.14 Troubleshooting

Common problems and their fixes

Symptom	Cause	Fix
`ModuleNotFoundError`	environment not activated	re-run the activate command from Step 2
`FileNotFoundError` on a data file	an earlier stage was skipped	run the stages in the documented order
Numbers differ from this book	a seed or parameter was changed	check the constants at the top of the generator
`command not found`	dependency missing from this env	`pip install -r requirements.txt` with the env active
Permission denied writing output	directory created by another user or process	delete the generated output directory and re-run

CHAPTER 13

The Payer Console

> *Five roles, one governed platform, and what the console refuses to show.*

13.1 Five roles in one product

15.1 The five target roles, exercised by one platform

Role	What they use
Data Analyst	the governed metric layer and its dimensions
Data Engineer	the warehouse, the semantic registry, the regression harness
QA / SDET	the validation suite and traceability matrix
ML Engineer	risk adjustment and hierarchical forecasting
AI Engineer	the constrained metric copilot

This was the brief for the final project: a single system that genuinely requires all five, rather than five components in one repository. A health plan works because the metric definitions are contractual, the models set payments, and the whole thing is audited.

13.2 What the console will not do

The interesting design choices are the refusals.

- **It will not answer an undeclared dimension.** The registry rejects it rather than returning something plausible.
- **It will not let the copilot write SQL.** The model resolves; the platform compiles.
- **It will not publish a definition change without its blast radius.** Thirty-three restated figures, named.
- **It will not report a fraud detector's baseline recall alone.** The adversarial scenarios sit beside it.

A GOVERNED PLATFORM IS DEFINED BY WHAT IT REFUSES

Every one of those refusals costs flexibility, and every one prevents a specific failure this book measured: a wrong-grain join, an injected query, a silent restatement, a security posture based on a number that collapses under adaptation.

The constraint is the product. An ungoverned platform can answer more questions and cannot tell you which of its answers to trust.

Running it

```
python run_all.py           # every stage in order
python app/api/main.py      # the console
```

CHAPTER 14

Results, and What They Do Not Establish

> *Every measured figure, and the limits of each.*

14.1 The numbers

16.1 Every measured figure

Metric	Value
Professional claim lines	124,508
Providers scored	900 (22 fraudulent)
Upcoding recall, baseline	**0.9545** (precision 0.913)
Upcoding recall, reduced_rate	**0.1818** (precision 0.5714)
Upcoding recall, stay_below / spread_thin	0.7727 / 0.5000
`impossible_day` rule hits	**0 in every scenario**
Metric snapshot values	50 across 10 specs
Figures restated by one definition change	**33 of 50**
Requirements traceability	11 of 12 (91.7%)
Members / events / censoring	24,000 / 15,446 / 35.64%
Median survival	11 months
Log-rank, individual vs employer	χ^2 = 187.08, p ≈ 0
Mean coefficient error: naive / Cox / hazard	**0.2965 / 0.1449 / 0.0113**
Competing risks: 1-KM vs CIF	**0.5053 vs 0.3996 (+0.1057)**
Risk adjustment: ridge / GB R^2	**0.1254** / 0.1127
Ridge calibration slope	1.0418 (aggregate error -0.735%)
Forecast MAPE: base / bottom-up / top-down	13.479% / 13.341% / 9.545%
Forecast MAPE: OLS / MinT	13.568% / 13.586% (worse than base)
Base forecast incoherence	690.60
Copilot: rules / LLM overall	0.875 / 0.250
Copilot LLM refusal accuracy	**0.000**

14.2 The six findings worth defending

1. The cheapest adaptation cost 77.3 points of recall

0.9545 to 0.1818 by upcoding 15% of claims instead of 42% - same scheme, same codes, same specialty. Both surviving rules are z-scores against a hard threshold, and a threshold is a cliff the adversary can sit below.

2. One definition change restated 33 of 50 published figures

Excluding pharmacy from loss ratio moved every loss-ratio slice materially, several across the 1.0 boundary that separates a profitable plan from an unprofitable one. The blast radius was computed and named rather than estimated.

3. The one untested requirement is the blast-radius publication itself

REQ-012 is the only gap in the traceability matrix, and it is the exact capability that produced finding 2. A matrix built to look complete would have reported 12 of 12.

4. The discrete-time hazard is 26x more accurate than a churn label

Mean coefficient error 0.0113 against 0.2965 naive and 0.1449 Cox - and it estimates a time-varying covariate neither of the others can express. 35.64% of members are censored, and a binary label lies about all of them.

5. 1-KM overstates voluntary lapse by 10.6 percentage points

0.5053 against a true CIF of 0.3996, a 26% relative overstatement, because treating death and plan-switching as censoring asserts those members could still have lapsed. The direction of this error is guaranteed, not incidental.

6. Top-down reconciliation beat OLS and MinT

9.545% MAPE against 13.568% and 13.586% - both of which are worse than the incoherent base forecast. The aggregate is far more forecastable than the leaves (4.27% against 16.68%), and 17 months is too little data to estimate MinT's covariance well.

14.3 What this does not establish

1. **That 18.18% is the floor.** Three adaptations were tried. An adversary combining them - reduced rate *and* avoiding the top code *and* spreading across providers - was not tested and would presumably do better.
2. **That the detector's baseline is meaningful at all.** 22 fraudulent providers out of 900. A 95% recall on 22 cases has a wide confidence interval, and one more miss moves it by 4.5 points.
3. **That top-down is generally best.** It won because the total is much more forecastable than the leaves in this hierarchy. With volatile aggregates and stable components it would lose, and MinT would recover its theoretical advantage with more training history.
4. **That R^2 of 0.125 is good or bad.** It is what healthcare cost prediction looks like at member level. The top-decile lift of 1.83 is the number a care-management programme would act on.
5. **That the injected coefficients resemble real churn drivers.** The 26x result is about estimator behaviour under censoring, not about insurance.
6. **That the semantic layer prevents definition drift.** It prevents *silent* drift. A change agreed in a meeting still happens - the platform's contribution is that its 33 consequences are enumerated before the reports go out.
7. **That any of this is production scale.** 24,000 members, 124,508 claim lines, one machine, one seed.

> **THE LAST WORD ON THE PORTFOLIO'S METHOD**
>
> Every project in this series has the same shape: build the obvious thing, build the cheap check that could embarrass it, and report what the check found. Fifteen times, the check found something the headline metric concealed.
>
> **None of those checks required more data, a better model, or more compute.** A held-out challenge set, a factorial design, a stratified breakdown, a red team, a per-series count, a calibration against ground truth. Each cost an afternoon. Each changed what the project was entitled to claim.

14.4 Fifteen projects, one method

This is the final volume, so it is worth stating the method the series converged on. It is not a methodology anyone set out with - it emerged because the same shape of mistake kept appearing.

16.2 Six checks, and the project where each earned its place

The check	Where it first paid	What it found
Build the trivial baseline first	Project 3	seasonal naive beat SARIMA on all three flagship series
Treat a perfect score as a bug report	Project 8	a regex scoring 1.000 against its own template
Stratify before concluding	Project 11	a fairness fix that helped on average and harmed thin files
Split on time, not at random	Project 13	PR-AUC 0.8495 against 0.0352 on identical features
Grade the grader	Project 14	a judge inflating a 22% agent to 100%
Let an adversary adapt	Project 15	recall 0.9545 falling to 0.1818

> **IN EVERY CASE THE HEADLINE METRIC WAS FINE AND THE SYSTEM WAS NOT**
>
> A 12.76% forecast MAPE, a 1.000 F1, a disparate impact of 0.887, a 0.8495 PR-AUC, a 100% agent score, a 95.45% detection recall. Every one of those numbers was correctly computed, and every one described something other than what a reader would take it to mean.
>
> **The failures were never in the arithmetic. They were in the question the arithmetic was answering** - a test set that shared the model's assumptions, an aggregate that spanned a boundary the system cared about, a scenario that assumed nobody would respond.

What that implies for reading anyone's results, including these

1. **Ask what the baseline was.** A number with nothing beside it cannot be evaluated. Five of these fifteen projects found the sophisticated method losing to the trivial one.
2. **Ask what would have made it come out differently.** If nothing would have, the measurement was decided before it ran.
3. **Ask which population, which period, which regime.** Four projects here found an aggregate concealing a finding that a split revealed.
4. **Ask who is on the other end.** Project 8's vision errors ran 8:2 against claimants; Project 11's mitigation harmed the applicants with the least credit history. Neither is visible in an accuracy figure.

THE ONE CLAIM THIS PORTFOLIO MAKES FOR ITSELF

Not that these fifteen systems are good. Several of them are measurably poor, and the books say so: a predictive-maintenance model at PR-AUC 0.035, a triage agent below a majority-class guess, a forecaster losing to naive on 40% of series.

The claim is that each number is traceable to a file produced by a run that actually happened, and that where the check disagreed with the hypothesis, the check was reported. That is the whole of it.

CHAPTER 15

Exercises, with Solutions

> *Problems requiring the chapters, with worked answers.*

15.1 Exercises

1. The detector's recall falls from 0.9545 to 0.1818 when upcoding drops from 42% to 15% of claims. Explain the mechanism using the z-score, and say why precision also falls.
2. One rule never fired in any scenario. Should it be removed? Argue both sides.
3. Excluding pharmacy from loss ratio moved 33 of 50 figures and left 17 unchanged. Explain which 17 and why.
4. The traceability matrix reports 91.7% coverage with REQ-012 uncovered. Why is that more useful than 100%?
5. 35.64% of members are censored. Explain how a binary churn label biases the premium-burden coefficient, and in which direction.
6. Name the coefficient the discrete-time hazard model estimates that neither logistic regression nor Cox can, and explain what makes it estimable.
7. 1-KM gives 0.5053 for voluntary lapse and the CIF gives 0.3996. Explain the assumption that fails, and say when the gap would be largest.
8. OLS and MinT reconciliation both scored worse than the incoherent base forecast. Explain why, and say what would have to change for MinT to win.

15.2 Solutions

1. The threshold cliff

Both surviving rules test $z > 2.5$ within specialty. Upcoding 42% of claims lifts the provider's mean intensity well past that; upcoding 15% lifts it roughly a third as far, leaving them below the line. **A threshold is a cliff, not a slope** - a provider at z = 2.4 is entirely invisible while one at z = 2.6 is flagged, and the fraud is nearly as profitable at 2.4. Precision falls from 0.913 to 0.5714 for the same reason: the genuine fraudsters have moved down into the z-range occupied by legitimate outliers, so the flags that do fire are now a mixture.

2. The rule that never fired

Keep it. `impossible_day` costs essentially nothing to evaluate and covers a real attack - crude volume fabrication - that simply is not present in this data. Removing it would leave that attack undefended for no saving. **Report it separately.** Including a rule with zero hits in a headline detection figure implies three signals are working when two are; an operator should know their coverage rests entirely on two correlated z-scores, both defeated by the same adaptation.

3. Which 17 did not move

The change affects `loss_ratio` only. The 17 unchanged figures are the `pmpm`, `denial_rate`, `member_months` and `paid_amount` slices, which use different definitions and different numerators. Every `loss_ratio` slice - total, by plan, by region, by month - moved, and all 33 moved materially. Note the useful part is not the count but the *names*: 33 specific published values, each of which can now be checked, reissued or footnoted.

4. Why 91.7% beats 100%

Because the gap is real and named. REQ-012 - 'a metric definition change publishes its blast radius' - has working machinery and no test asserting it works, which to an auditor is indistinguishable from a capability that happens to work today. A matrix reporting 12 of 12 would either be wrong or would have been generated to look complete. **An honest gap is actionable; an unexamined 100% is not evidence of anything.**

5. How censoring biases the coefficient

8,554 members had not lapsed when observation ended and are labelled 'retained'. Many have high premium burden and would have lapsed given more time. The model therefore sees high-burden members in *both* classes and compensates by inflating the coefficient to separate the ones who did lapse - which were the earliest and most extreme cases. Measured: **1.5698 against a true 0.95, 65% too large**, and an upward bias in general.

6. The time-varying coefficient

`tenure_at_month`, true value -0.03, estimated at -0.0188. It changes every month a member remains enrolled, so a binary logistic model has no row to attach it to and standard Cox assumes time-fixed covariates. **Person-month expansion makes it estimable**: reshape to one row per member per month at risk and fit $h(t|x) = \sigma(\alpha_t + \beta^\top x_t)$. Because each row is a month, any covariate may vary by row - the reshaping costs only rows and buys the entire class of time-varying covariates.

7. The competing-risks assumption

Censoring assumes the member *could still experience the event later*. A deceased member cannot voluntarily lapse; neither can one who already switched plans. Treating them as censored asserts an impossible counterfactual, so 1-KM survival-weights the lapse hazard over a population that no longer exists - always an overstatement. **The gap is largest where the competing share is largest**: here 37.9% of exits are non-voluntary, giving +0.1057. In a senior product with high mortality the error would be considerably worse.

8. Why the optimal methods lost

MinT minimises the trace of the forecast error covariance and is provably optimal *given correct covariances*. With 17 training months and 24 series, that covariance is estimated from very little data, and badly estimated weights are worse than no weighting - the same failure as Project 12's IPS at an effective sample fraction of 0.176. Top-down wins because the total is far more forecastable than the leaves (4.27% against 16.68% MAPE), so forecasting only the aggregate avoids inheriting leaf noise entirely. **For MinT to win you would need more training history**, or a hierarchy where the leaves are not dramatically noisier than the total.

CHAPTER 16

Appendix

> *Repository map, glossary, and reproducibility notes.*

16.1 A - Repository map

Where everything lives

Path	Purpose
platform/ingest/generate_payer_data.py	the seeded payer book and every injected mechanism
platform/warehouse/build_warehouse.py	the warehouse
platform/semantic/metrics.py	the metric registry and SQL compiler
integrity/metric_regression.py	golden values and the blast-radius computation
integrity/upcoding_detection.py	three signals and the red team
analytics/survival/member_survival.py	KM, log-rank, three estimators, competing risks
analytics/risk/risk_adjustment.py	time-split cost prediction and calibration
analytics/forecasting/hierarchical_forecast.py	four reconciliation approaches
ai/metric_copilot.py	the constrained copilot
qa/validation.py	the requirements traceability matrix
app/api/main.py	the payer console
reports/upcoding_detection.json	the baseline and every adversarial scenario
reports/metric_regression.json	golden drift and the 33 restated figures
reports/survival.json	curves, estimators, competing risks and the injected truth
reports/metric_copilot.json	every question, resolution and refusal
reports/validation.json	the traceability matrix and its gap

16.2 B - Glossary

Terms used in this book

Term	Meaning
Seeded generator	a synthetic data producer whose random draws are reproducible from a fixed starting value
Ground truth	the mechanism deliberately injected into synthetic data, used only to score an analysis - never to make a decision
Control arm / baseline	a simple alternative a new method must beat before it is worth its complexity
Point-in-time correctness	computing a feature using only information that existed at the moment being modelled
Leakage	information from the future or from the target entering a feature, inflating every offline metric
Reproducibility checksum	a hash of generated output, compared across a fresh process to prove determinism
Loss ratio	claims paid divided by premium collected; above 1.0 the plan loses money
PMPM	per member per month - cost normalised so it does not confound price with membership
Semantic layer	one governed definition per metric, with declared dimensions
Metrics-as-code	metric definitions kept in version control and compiled, not hand-written
Metric regression test	asserting that published figures are computed the same way as before
Blast radius	the specific published figures a definition change restates
Traceability matrix	the mapping from requirements to specifications to tests
Censoring	an observation ending before the event of interest occurs
Kaplan-Meier	the non-parametric product-limit estimator of a survival curve
Greenwood's formula	the variance estimator for a Kaplan-Meier curve
Log-rank test	comparing two survival curves across their whole length
Cox proportional hazards	a semi-parametric model of the hazard ratio
Discrete-time hazard	logistic regression over person-month rows; admits time-varying covariates
Person-month expansion	reshaping to one row per subject per period at risk
Competing risk	an event that prevents the event of interest from ever occurring
Cumulative incidence function	the honest cumulative probability of one cause among several
Risk adjustment	predicting expected cost so payments reflect population risk
Calibration slope	the regression of actual on predicted; 1.0 means correct at every level
Top-decile lift	how much more the top-ranked tenth costs than average
Hierarchical forecasting	forecasting a hierarchy so the parts sum to the whole
Coherence	forecasts that add up across the hierarchy
MinT	minimum-trace reconciliation; optimal given correct error covariances
Upcoding	billing a higher-paying code than the service justifies
Red team	re-evaluating a control against an adversary who adapts to it

16.3 C - Reproducibility

Every dataset in this project is produced by a seeded generator, so the whole pipeline can be rebuilt from nothing and should produce byte-identical output.

> **A FIXED SEED IS A CLAIM, NOT A GUARANTEE**
>
> Two projects in this portfolio shipped generators with fixed seeds that were still not reproducible across runs. The causes were `rng.choice(list(a_set))` and `abs(hash(some_string))` - both of which are perfectly stable *within* one process and vary between processes, because Python randomises string hashing per interpreter run.
>
> Neither is visible from reading the code, from the seed, or from the output looking sensible. Both were caught only by hashing the generated files and comparing across a *fresh* process. The defences used throughout this portfolio are to sort any collection before it feeds a random draw, and to verify by checksum in a new process rather than trusting the seed.

16.4 D - Where this leads next

This is the last project in the series, so the thread ends here - but the arguments it closes were opened much earlier. **Project 2** modelled churn as a binary label and flagged censoring as unhandled; Chapter 6 shows that label is 26 times less accurate than the alternative. **Project 3** forecast a hierarchy bottom-up and named reconciliation as missing; Chapter 11 implements four methods and finds the two optimal ones lose. **Project 6** derived that a fixed threshold gives an adversary a safe region; Chapter 10 measures the cost of exploiting it. **Project 9** let a model write SQL and guarded it syntactically; Chapter 12 removes the attack surface instead. **Project 13** measured what a random split does to a model; Chapter 8's split is temporal by design, with the reason written into the artefact.

The three most valuable improvements to *this* project, in order: replace the cliff-edge z-score thresholds with a graduated response, since a hard threshold is what makes the 15%-upcoding adaptation profitable; write the test behind REQ-012, because the blast-radius publication is the platform's most consequential capability and is the one thing nothing verifies; and test a *combined* adversarial scenario, since three adaptations were measured separately and an opponent would use all three at once.